



A framework for modeling and analyzing cyber-physical systems using statistical model checking

Abdel-Latif Alshalalfah, Otmane Ait Mohamed, Samir Ouchani

► To cite this version:

Abdel-Latif Alshalalfah, Otmane Ait Mohamed, Samir Ouchani. A framework for modeling and analyzing cyber-physical systems using statistical model checking. Internet of Things, 2023, 22, pp.100732. <10.1016/j.iot.2023.100732>. <hal-04108550>

HAL Id: hal-04108550

<https://hal.science/hal-04108550v1>

Submitted on 11 Jan 2024

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

A Framework for Modeling and Analyzing Cyber-Physical Systems using Statistical Model Checking

Abdel-Latif Alshalalfah
Otmane Ait Mohamed · Samir Ouchani

Received: date / Accepted: date

Abstract The trustworthiness of a cyber-physical system is essential for it to be qualified for utilization in most real-life deployments. This is especially critical for systems that deal with precious human lives. Although these safety-critical systems can be investigated using both experimental testing and model-based verification, accurate models have the potential to permit risk-free mimicking of the system behavior even in the most extreme scenarios. To overcome the CPS modelling and design challenges, the INCOSE/OMG standard System Modeling Language (SysML) is utilized in this work to accurately specify cyber-physical systems. For that, a bounded set of SysML constructs are defined to precisely capture the semantics of continuous-time and discrete-time system behaviors. Then, the SysML constructs are substituted by developing a new algebra, called Enhanced Activity Calculus (EAC). So, EAC helps construct equivalent priced timed automata models by developing a new systematic procedure to correctly translate the SysML models into the statistical model checking tool UPPAAL-SMC inputs. The latter checks whether the system is correct and safe or not. Moreover, the soundness of the developed translation mechanism has been proved and its effectiveness has been shown on a real use case, namely the artificial pancreas.

Keywords System Modeling Language · Enhanced Activity Calculus · Cyber-Physical Systems · Model Transformation · Model-Based Verification · Safety-Critical · Statistical Model Checking · Priced Timed Automata

A.-L. Alshalalfah
Department of Electrical & Computer Engineering, Concordia University, Montreal, Canada.
E-mail: abdellatif.alshalalfah@mail.concordia.ca

O. Ait Mohamed
Department of Electrical & Computer Engineering, Concordia University, Montreal, Canada.

S. Ouchani
CESI Lineact, Aix-en-Provence, France.

Nomenclature

<i>CPS</i>	Cyber-Physical Systems
<i>EAC</i>	Enhanced Activity Calculus
<i>HA</i>	Hybrid Automata
<i>MITL</i>	Metric Interval Temporal Logic
<i>ODE</i>	Ordinary-Differential Equations
<i>ODESCD</i>	ODE of SysML Constraint Diagram
<i>PID</i>	Proportional-Integral-Derivative
<i>PTA</i>	Priced Timed Automata
<i>SMC</i>	Statistical Model Checking
<i>SysML</i>	Systems Modeling Language

1 Introduction

Whether human-operated or autonomous, embedded systems are designed to improve the quality of life for people. From embedded computing to distributed systems, Cyber-Physical Systems (CPS) refer to computing systems that interact with control and management objects [53]. As technology advances, CPS is being used in a wide range of applications [30]. With the reduction in size and cost of hardware, along with accelerating innovation and advancement in sensor and computational technologies, CPS has been able to spread to all types of applications. Through horizontal expansion, CPS has gained popularity in all types of application. Also, CPS flourished vertically to find a foot in more complex and dependable applications. From daily applications, the various success stories have encouraged designers to develop CPS for autonomous control compared to the early systems which required some degree of human interaction [23, 41]. Nowadays, wireless body area networks are utilized to connect devices that observe the status of the physiological dynamics [16]. As a result, health conditions can be monitored and treated in a timely manner. Patients with chronic diseases will particularly benefit from this. For example, with around half a billion diabetes worldwide [42], an automatic glucose controller is necessary for them to live a normal life while still avoiding the health complications related to their situation.

In order to get approval certificates from the appropriate authorization entities, these systems must prove their safety and robustness under all scenarios [11]. However, for real-life deployments, only qualified systems must meet these safety requirements. From the first prototype to the final fabricated product, verifying the safety of CPS is a vital step in the development process. The system-level analysis provides feedback early in the design process, and by identifying safety issues early, time and resources are not wasted [29]. Additionally, the system-level analysis helps understand CPS limitations and define the requirements of CPS components for safe operation. Furthermore, CPS can be verified under extreme scenarios that would be impossible

to conduct in real life without taking extraordinary risks by using appropriate realistic models.

Analyzing systems at the system level is either accomplished through simulation testing or through formal methods. In the former approach, specific input scenarios can be used to evaluate CPS behavior. Yet, it does not give confidence on the state space coverage. On the other hand, formal methods such as model checking [10] provide exhaustive coverage for the whole state space. Unfortunately, formal techniques do not scale well for realistic hybrid systems and suffer from the infamous state space explosion problem [21].

As a compromise between these two approaches, Statistical Model Checking (SMC) can be used for verification. Although it does not provide exhaustive coverage for the state space, SMC can be used to introduce statistical guarantees for safety properties with feasible computational resources. In a nutshell, the following are the main contributions of this work.

- Proposing a novel systematic procedure to capture the semantics of SysML-based diagrams and to construct its equivalent PTA models for SMC analysis.
- The effectiveness of the proposed framework to analyze a medical CPS is demonstrated on an artificial pancreas case study. In particular, the safety of the system is verified using SMC to evaluate the ability of three control configurations to mitigate message errors.

Below is an outline of the remainder of the article. The literature review is presented in Section 2, and then Section 3 demonstrates, through an artificial pancreas example, the SysML graphical and textual modeling. Afterwards, Section 4 introduces the new proposed automatic construction of equivalent Priced Timed Automata (PTA) models and proves the soundness of the developed approach. Section 5 illustrates the experiments conducted for model validation and safety verification procedure by an example experiment, and section 6 concludes the article.

2 Literature Review

With the growing demand for CPS applications, several research works have investigated the verification and safety analysis problems related generally to CPS. Based on our surveyed initiatives, we have identified two main categories: *Formal verification* and *Simulation* based approaches.

2.1 Simulation based approaches

Even before the advent of modern computer systems, the term *Simulation* is known as the process of designing a model of a real system to conduct experiments [52]. These experiments aim at understanding the system's behavior or evaluating a strategy associated with the system. Simulation software tools

have flourished with the advent and availability of low-cost computational systems.

Liu et al. [37] have used the open-source toolkit MATSim [55] to investigate large-scale transportation patterns for shared autonomous vehicles. In their work, agent-based modelling is applied to estimate mode choices between human-driven vehicles, shared autonomous vehicles, and public transit. Following a cost function that takes into account, the out-of-pocket, the trip time, and the waiting time, each driver chooses one of the three options of travel mode. The analysis is done for different fare levels, demographic settings, and shared autonomous vehicles availability to give implications on sustainability.

In [32], an assessment of the safety of leader-follower configurations for autonomous radar semi-trucks is made based on different environmental conditions. The simulation model is developed with the commercial platforms AmeSim, PreScan, and Matlab-Simulink to study the effect of environmental conditions on safety margins in semi-truck convoy platooning. The autonomy in their simulated vehicles is enabled by adopting sensors for radar, global positioning systems, and short-range inter-vehicle communication.

Instead of fully autonomous vehicles, the work in [5] addressed semi-autonomous vehicles implementing adaptive cruise control coexisting with regular vehicles and trucks. The vehicles enter the four-lane highway with a user-predefined arrival rate in the microscopic Java-based F.A.S.T. traffic simulator. Their findings show that a high penetration of semi-autonomous vehicles can increase traffic performance, especially under high traffic conditions.

Connected and autonomous vehicles and their impact on road safety are discussed in [47]. Initially, the simulation software VISSIM is utilized to study a test-bed that mimics a three-lane motorway with traffic statistics measured from a real one in England. A lateral and longitudinal control algorithm is then tested for its ability to reduce traffic conflicts at different market penetration rates.

From a healthcare perspective, a falsification approach is presented in [48, 9] to simulate and verify the artificial pancreas controller in a simulation environment. The S-Taliro tool which applies falsification simulations terminates with either finding a safety violation or failure to find, without the explicit guarantee that such one does not exist. Instead, the tool uses robustness metric to predict the distance between simulation outcomes and safety margins.

2.2 Formal based approaches

Unlike the numerical simulation approaches which mimic the behavior of real systems, formal methods apply analytical reasoning to derive mathematically-proven properties that characterize the system behavior. These characteristics are not always attainable, but when achieved they provide guaranteed outcomes which is an asset that helps verify safety-critical systems.

In [28], piece-wise affine hybrid automata was used to analyze the wind turbine dynamics in SpaceEx verification platform [20]. Even though Kekatos et al. reduced some blocks for better scalability, the resulting model contained around 16 million locations, which would hinder the ability to analyze more elaborate systems. However, classical hybrid automata (HA) tools and methodologies suffer from this limitation [49].

The problem of formally analyzing a swarm of robots is handled by Schupp et al. [50]. The cooperative decentralized robots are modeled as a hybrid system and investigated by *flowpipe* analysis where the sets of reachable states are iteratively over-approximated [19]. Although the work in [50] deals with a simple model of distributed synchronization, it still causes some scalability challenges that are partially encountered by compositional analysis and optimized transition emulation.

Using a combination of simulations and formal analysis, [46] examines patient-controlled analgesia's safety. So, to analyze the resulting CPS, its detailed behavior is modeled in Simulink. Then, to qualify the CPS for model checking, the continuous dynamics are abstracted away from the system model and then replaced by simple timing constraints with the target to be analyzed in UPPAAL model checker [7]. Additionally, UPPAAL is also used in [26] to verify control algorithms in a dual chamber implantable pacemaker. Meanwhile, a timed automata representation of the heart and the pacemaker are used to specify the ability of the algorithms to avoid unsafe regions of the state space. The proposed approach covers the whole state space, yet only the state space that is modeled. Thus, this excludes certain control and physiological behaviors that are beyond the expressive power of the modeling language and the computational feasibility of the verification technique. In fact, these behaviors can be skipped in some systems but are essential to correctly analyze hybrid systems with continuous-time variables.

2.3 Statistical model checking based approach

SMC consists of observing a number of simulation runs or system executions and using statistical methods to reason about formal properties [35].

After some preliminary works such as the hypothesis testing of modal properties in process algebra [33], initial results for SMC had witnessed progress since 2002 [58] with the corresponding term introduced for the first time in 2004 [51]. Reasoning about reachability problems with SMC algorithms provides mainly guarantees on the probability error bound. Depending on the type of reachability expression being dealt with, the error bound can be calculated by utilizing the appropriate classical mathematics such as Monte Carlo with Chernoff-Hoeffding error bounds [43, 24] or hypothesis testing using Wald's sequential analysis [56].

Different tools exist that implement SMC algorithms such as PRISM [31], UPPAAL-SMC [12, 15], BIP [39], and Ymer [57]. Since their inception, SMC tools have been utilized to study many discrete-time and continuous-time sys-

tems. To list a few: airplane cabin communication system [6], distributed sensor network [36], energy-aware house heating [13], biological mechanisms of the genetic oscillator [14], real-time streaming protocol [44], artificial pancreas [2, 4], anesthesia control [3], and coordinated emergency braking system [1].

2.4 Model Construction

In order to analyze the system, it is necessary to first convert the specifications into the modeling language used by the analysis tool. Furthermore, an adequate level of expertise is required to model the system properly when done manually. Furthermore, formal modeling languages tend to be more error-prone due to their low readability. Therefore, the need arises to facilitate the process of constructing formal models by automatically translating high-level models that incur better readability.

In [28], the system modeled in Simulink is translated into SpaceEx modeling language in four steps. After the Simulink model is modified to comply with the verification standards, the tool SL2SX [40] is employed to handle the main translation step and construct a SpaceEx model. Afterwards, compositional syntactic hybridization [27] and validation are conducted to achieve a model ready to be analyzed.

An approach to transform Simulink models into UPPAAL-SMC is proposed in [18]. The work is employed on two automotive use cases for brake-by-wire and an adjustable speed limiter. The Simulink models are first reduced by the flattening procedure. Then, each block is replaced by an equivalent timed automaton composed of three locations: start, offset, and operate. Still, their approach does not implement complex real-valued blocks in UPPAAL-SMC but addresses them in Simulink instead.

Instead of commercial modeling tools, System Modeling Language (SysML) [54] can be used to specify CPS. SysML is the defacto standard modeling language for systems engineering with rich semantics and expressive power sufficient to describe system structures and behaviors at various levels of abstraction [25]. Ouchani et al. [45] constructed probabilistic automata by converting SysML models. The resulting models were incurred to analyze security properties of the real-time streaming protocol using the probabilistic model checker PRISM [31].

Compared to the studied initiatives, the main objective of this work is to develop a framework that enables efficient modeling and analysis for CPS. The proposed framework takes system behavior specified using SysML diagrams as input. The novelty of this proposed work is summarized by the following contributions.

- Defining a bounded set of SysML constructs that are sufficient to capture the behaviors of the CPS discrete-time and continuous-time dynamics.
- Defining textual specification language for SysML by extending the semantics initially developed in [17, 45].

- Proposing a novel systematic procedure to transform the SysML behavioral specifications into PTA. Compared with the previous works that processed models specified in the commercial tool Simulink [28, 40, 18] or did not support modeling physical processes [18, 45], this new proposed approach defines a systematic procedure to process SysML models for the CPS and to construct an equivalent PTA model for analysis by supporting more features and expressive powers to specify physical properties like *time*, *rate* and *real-numbers related measurements*.
- The soundness of the proposed approach has been proven and its effectiveness to analyze CPS is demonstrated on an artificial pancreas system.

3 The Proposed Framework

Fig. 1 provides a brief overview of the proposed framework that runs on the following steps.

- ① The process starts with the initial identification of the CPS to explore the nature of its application. This step helps specify the system's requirements including the safety properties that have to be met.
- ② The topology of the system is defined by specifying the functional components of the system which are used to create the SysML block definition diagram. Also, the interactions between the CPS components are used to define the SysML flow diagram. Integrated CPS are formed from continuous real-time components describing physical processes and discrete-time components describing cyber processes.
- ③ By relying on the existing topologies, behavioral models for the physical components are imported in the form of Ordinary-Differential Equations (ODE). Similarly, the cyber components of the system are imported from design specifications in the form of discrete variations.
- ④ Physical and cyber components are represented using SysML parametric constraint diagrams and activity diagrams, respectively.
- ⑤ To automate further processing, each of the SysML diagrams are written in textual format. For a constraint diagram describing the physical dynamics, the representation is done using the proposed syntax named Ordinary-Differential Equations of SysML Constraint Diagram (ODESCD). For an activity diagram describing a component's behavior, the representation is done using the proposed Enhanced Activity Calculus (EAC).
- ⑥ A new systematic algorithm is proposed to convert ODESCD and EAC blocks into equivalent PTA blocks. The SysML block definition diagram, describing the system's structure, specifies the input/output connections of each PTA block.
- ⑦ The various PTA blocks for physical dynamics and component behaviors are mapped as described by the flow diagram. The parallel composition of all the PTA blocks form the integrated CPS that is processed.

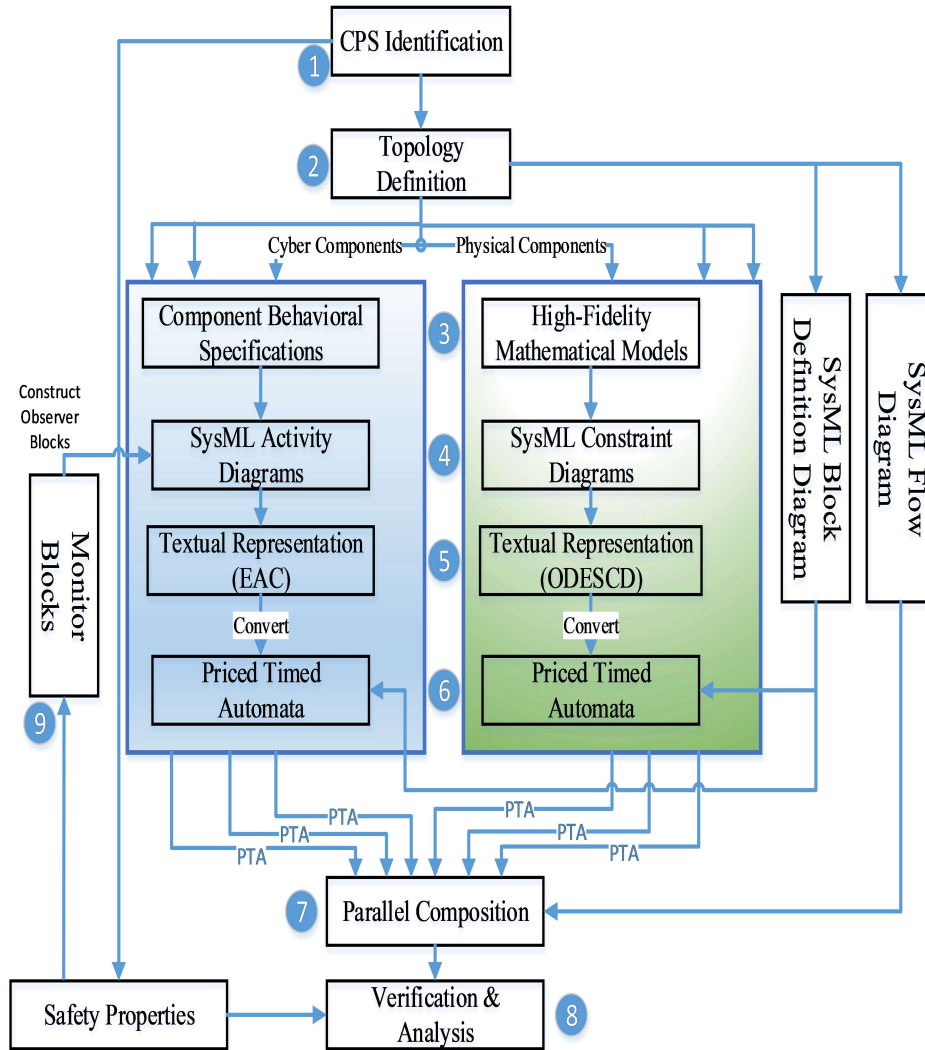


Fig. 1: The Proposed Framework Workflow.

- ⑧ The analysis tool UPPAAL-SMC is used to analyze the system and verify the safety properties. The framework is demonstrated on an artificial pancreas system alongside a proposed representation of continuous-time and discrete-time dynamics.
- ⑨ For safety properties that are beyond the expressive power of the query language in use, dedicated monitor blocks are employed to observe specific

phenomena. An observer block is then added to the system by developing a behavioral model for that block which is specified using a SysML activity diagram. The new block is then processed as component of the CPS to construct an equivalent PTA model. By adding these monitor blocks, more complex safety properties are simplified and expressed easily in order to be examined for safety.

3.1 SysML Graphical and Textual Modeling

3.1.1 SysML for *continuous-time dynamics*

The dynamics of physical processes describe the flow of physical quantities in the real world. These quantities are represented by real-valued real-time variables where the derivative of a variable is equivalent to the change on its associated physical quantity. Therefore, it is common for continuous-time dynamics to be specified by a system of ODEs. SysML constraint diagrams can be used to model ODEs.

Notation 1 (ODE of SysML Constraint Diagram (ODESCD)). ODESCD is defined as a tuple $(X, X^0, K, P, R, F, I, O)$, where:

- X is a set of real-time real-valued differentiable variables,
- X^0 is a set of initial values,
- K is a set of real-valued equation coefficients,
- P is a set of constant real-valued parameters,
- R is a set of real-time real-valued variables,
- $F(X, R)$ is a set of real-valued functions,
- $I \in X \cup K \cup P \cup R$ is a set of input variables, and
- $O \in X$ is a set of output variables.

Definition 1 (Semantics of ODESCD). Let $(X, X^0, K, P, R, F, I, O)$ be a ODESCD, its semantics is defined as the dynamics of a physical system described by a set of ODEs as follows (in this context a subscript in the form of $a_1 \times a_2$ indicates the matrix dimensions).

$$X'_{n \times 1}(t) = K_{n \times n}(X, P, R, t) X_{n \times 1} + F_{n \times 1}(X, R) \quad (1)$$

$$X_{n \times 1}(t = 0) = X^0_{n \times 1} \quad (2)$$

$X'_{n \times 1} = [x_1 \ x_2 \ \dots \ x_n]$ is the set of differential variables to be solved, $X^0_{n \times 1} = [x_1^0 \ x_2^0 \ \dots \ x_n^0]$ is the set of initial values for the differential variables, $K_{n \times n}(X, P, R, t)$ is the set of differential equation coefficients which can be constants or functions of constant parameters, real-time variables or time, P is the set of additional constant parameters for the equation, R is the set of additional real-time variables, $F_{n \times 1}(X, R)$ is the additional terms of the ODE, $I \in X \cup K \cup P \cup R$ is the set of input variables which can be parameters or real-time variables, and $O \in X$ is the set of output variables which is a subset of the ODE solution.

316 In this system, I is **defined** to utilize variables and parameters that are
 317 **provided as** input to the ODESCD definition, and O is used to export the
 318 desired variables from the solution of ODESCD.

❖ **ODESCD example: meal glucose absorption model**

X is a vector representing carbo-hydrate measures in the stomach where Q_{sto1} and Q_{sto2} are the stomach glucose amounts in solid state and liquid state, respectively, and rag is the blood glucose rate of appearance. These physical quantities are initially nulled as assigned in X^0 . Fig. 2 depicts the SysML constraints block diagram for meal absorption variations measures.

$$X = [Q_{sto1} \ Q_{sto2} \ rag]^T \quad (3)$$

$$X^0 = [0 \ 0 \ 0]^T \quad (4)$$

$$[K] = \begin{bmatrix} -k_{gri} & 0 & 0 \\ k_{gri} & -k_{empt}(Q_{sto1}(t) + Q_{sto2}(t), D_{meal}) & 0 \\ 0 & \frac{f \cdot k_{abs}}{BW} k_{empt}(Q_{sto1}(t) + Q_{sto2}(t), D_{meal}) & -k_{abs} \end{bmatrix}$$

$$k_{empt}(Q, D_{meal}) = \begin{cases} k_{min} + \frac{k_{max} - k_{min}}{2} (\tanh(\alpha(Q - b \cdot D_{meal})) - \tanh(\beta(Q - c \cdot D_{meal})) + 2) & D_{meal} > 0 \\ 0 & D_{meal} = 0 \end{cases}$$

$$\alpha = \frac{5}{2 \cdot D_{meal} \cdot (1 - b)} \quad (5)$$

$$\beta = \frac{5}{2 \cdot D_{meal} \cdot c} \quad (6)$$

$$P = \{k_{gri}, k_{abs}, f, BW, b, c, k_{min}, k_{max}\} \quad (7)$$

$$R = \{cur_Meal, D_{meal}\} \quad (8)$$

$$F(X, R) = [cur_Meal(t) \ 0 \ 0]^T \quad (9)$$

$$I = \{cur_Meal, D_{meal}\} \quad (10)$$

$$O = \{rag\} \quad (11)$$

❖ **ODESCD example: glucose-insulin dynamics**

X is a vector representing the various physical quantities for the glucose and insulin dynamics all over the body compartments. I_{sc1} and I_{sc2} are the insulin levels in the subcutaneous tissues, X_1 is the insulin in the interstitial fluid, $\{G, G_s, G_t\}$ are the glucose levels in the blood, subcutaneous tissues, and slowly equilibrating tissues respectively. I_p is the plasma insulin, I_l is the portal vein insulin, and I_d is the delayed insulin signal. These physical quantities are initialized as in the vector X^0 . Fig. 3 depicts the SysML Constraint Block diagrams for Glucose-Insulin variations measures.

$$X = [I_{sc1} \ I_{sc2} \ X_1 \ G_s \ I_1 \ I_d \ I_l \ I_p \ G \ G_t]^T \quad (12)$$

$$X^0 = [I_{sc1_{ss}} \ I_{sc2_{ss}} \ 0 \ G_i \ I_b \ I_b \ I_{lb} \ I_{pb} \ G_i \ G_{ti}]^T \quad (13)$$

«constraint»	
Meal Absorption : Equality	
$Q_{sto1}' == -k_{gri} \cdot Q_{sto1} + cur_Meal(t)$ $Q_{sto2}' == -k_{empt}(Q_{sto1}+Q_{sto2}, D_{meal}) \cdot Q_{sto2} + k_{gri} \cdot Q_{sto1}$ $rag' == -k_{abs} \cdot rag + f \cdot k_{abs} \cdot k_{empt}(Q_{sto1}+Q_{sto2}, D_{meal}) \cdot Q_{sto2} / BW$	
Parameters: Input: $cur_Meal [mg/min]$ (real time glucose intake) , $D_{meal} [g]$ (meal carbs) Output: $rag [mg/Kg/min]$ (real time glucose rate of appearance in the blood)	

Fig. 2: SysML Constraint Block for Meal Absorption

$$[K] = \begin{bmatrix} -(k_d + k_{a1}) & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ k_d & -k_{a2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -p_{2u} & 0 & 0 & 0 & 0 & \frac{p_{2u}}{V_I} & 0 & 0 \\ 0 & 0 & 0 & -\frac{1}{T_s} & 0 & 0 & 0 & 0 & \frac{1}{T_s} & 0 \\ 0 & 0 & 0 & 0 & -k_i & 0 & 0 & \frac{k_i}{V_I} & 0 & 0 \\ 0 & 0 & 0 & 0 & k_i & -k_i & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -(m_1 + \frac{m_6 \cdot m_1}{1-m_6}) & m_2 & 0 & 0 \\ k_{a1} & k_{a2} & 0 & 0 & 0 & 0 & m_1 & -(m_2 + m_4) & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{-k_{p3}}{V_G} & 0 & 0 & -(k_{p2} + k_1) \frac{k_2}{V_G} & \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & K_1 \cdot V_G & -k_2 \end{bmatrix}$$

$$P = \{I_{sc1ss}, I_{sc2ss}, G_i, I_b, I_{pb}, G_{tb}, G_b, k_d, k_{a1}, k_{a2}, p_{2u}, V_I, I_b, T_s, k_i, V_G, m_1, m_6, m_2, m_4, \\ k_{p1}, k_{p2}, k_{p3}, F_{cns}, k_{e1}, k_{e2}, k_1, k_2, V_{m0}, V_{mx}, K_{m0}, K_{mx}\} \quad (14)$$

$$R = \{rag, IIR\} \quad (15)$$

$$F(X, R) = [IIR, 0, -p_{2u} \cdot I_b, 0, 0, 0, 0, 0, 0, \frac{rag + k_{p1} - F_{cns}}{V_G} \\ - k_{e1} \cdot max(0, G - \frac{k_{e2}}{V_G}), -\frac{(V_{m0} + V_{mx} \cdot X_1) \cdot G_t}{K_{m0} + K_{mx} \cdot X_1 + G_t}]^T \quad (16)$$

$$I = \{rag, IIR\} \quad (17)$$

$$O = \{G_s, G\} \quad (18)$$

«constraint» Glucose-Insulin Dynamics : Equality
$I_{sc1}' = -(k_d + k_{a1}) \cdot I_{sc1} + IIR$ $I_{sc2}' = k_d \cdot I_{sc1} - k_{a2} \cdot I_{sc2}$ $X_1' = -p_{2u} \cdot X_1 + p_{2u} \cdot (I_p / V_l - I_b)$ $G_s' = -(G_s - G) / T_s$ $I_1' = -k_i \cdot (I_1 - I_p / V_l)$ $I_d' = -k_i \cdot (I_d - I_1)$ $I_l' = -(m_1 + (m_6 \cdot m_1 / (1 - m_6))) \cdot I_l + m_2 \cdot I_p$ $I_p' = -(m_2 + m_4) \cdot I_p + m_1 \cdot I_l + k_{a1} \cdot I_{sc1} + k_{a2} \cdot I_{sc2}$ $G' = (k_{p1} - k_{p2} \cdot G \cdot V_G - k_{p3} \cdot I_d - F_{cns} - k_{e1} \cdot \max(0, G \cdot V_G - k_{e2}) - k_1 \cdot G \cdot V_G + k_2 \cdot G_t + rag) / V_G$ $G_t' = -(G_t \cdot (V_{m0} + V_{mx} \cdot X_1)) / (K_{m0} + K_{mx} \cdot X + G_t) + k_1 \cdot G \cdot V_G - k_2 \cdot G_t$
Parameters: Input: IIR [pmol/Kg/min] (subcutaneous insulin infusion rate) Input: rag [mg/Kg/min] (meal glucose rate of appearance in the plasma) Output: G_s [mg/dL] (real time subcutaneous glucose level) Output: G [mg/dL] (real time blood glucose level)

Fig. 3: SysML Constraint Block for Glucose-Insulin Dynamics

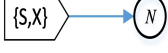
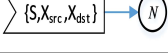
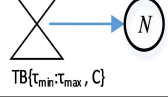
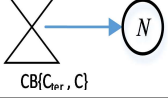
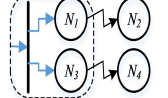
3.1.2 SysML for discrete-time dynamics

Discrete-time dynamics are described by SysML activity diagrams. So, [in order to precisely describe CPS and capture exactly its underlying semantics](#), we develop Enhanced Activity Calculus (EAC) to formally describe SysML activity diagrams by extending NuAC presented in [17, 45]. These enhancements include redefining existing nodes as well as proposing new nodes for time-bounded delay, constraint-bounded delay, and competing events. The list of the used activity nodes and their textual EAC representation is shown in Table 1.

❖ EAC example: artificial pancreas

The artificial pancreas is composed of a sensor (Fig. 4) that periodically measures the glucose level, sends it over wireless channel (Fig. 5) to the controller. [Then, the](#) controller (Fig. 6) calculates the required amount of insulin, and the actuator (Fig. 7) applies the control action. Lastly, the SysML activity diagram describing the meal scenario is shown in Fig. 8.

Table 1: SysML Enhanced Activity Calculus Nodes Syntax.

SysML Term	SysML Activity Diagram Structure	EAC Syntax
Activity Initial Node		$l \mapsto N$
Action Node		$l : ACT(A) \mapsto N$
Call Procedure		$l : CALL_P(A) \mapsto N$
Send Node		$l : \{S, X\} ! \mapsto N$
Receive Node		$l : \{S, X_{src}, X_{dst}\} ? \mapsto N$
Merge Node		$l : Mrg \mapsto N$
Guarded Branch		$l : BC(l_{i_1} : (C = C_1) \mapsto N_1, \\ l_{i_2} : (C = C_2) \mapsto N_2, \dots)$
Probabilistic Branch		$l : BP(l_{i_1} : (P = P_1) \mapsto N_1, \\ l_{i_2} : (P = P_2) \mapsto N_2, \dots)$
Time-Bounded Delay Node		$l : DTB(\tau_{min} : \tau_{max}, C) \mapsto N$
Constraint-Bounded Delay Node		$l : DCB(C_{ter}, C) \mapsto N$
Competing Events		$l : Comp_Events(N_1 \mapsto N_2, \\ N_3 \mapsto N_4, \dots)$

By substituting the SysML nodes with their textual equivalents following Table 1, the EAC representation of these activity diagrams is shown below.

$$Act_Sensor = l \mapsto l_1 : Mrg \mapsto N_1$$

$$N_1 = l_2 : DTB(T_p,) \mapsto l_3 : ACT(meas_var = phy_var) \mapsto l_4 : \{S_{et}, meas_var\} ! \mapsto l_1$$

$$Act_Channel_{lossy} = l \mapsto l_1 : Mrg \mapsto N_1$$

$$N_1 = l_2 : \{S_{et1}, var_in, var_out\} ? \mapsto l_3 : BP(l_4 : (P = P_S) \mapsto N_2, l_5 : (P = P_F) \mapsto l_6 : Mrg \mapsto l_1)$$

$$N_2 = l_7 : \{S_{et2}, var_out\} ! \mapsto l_6$$

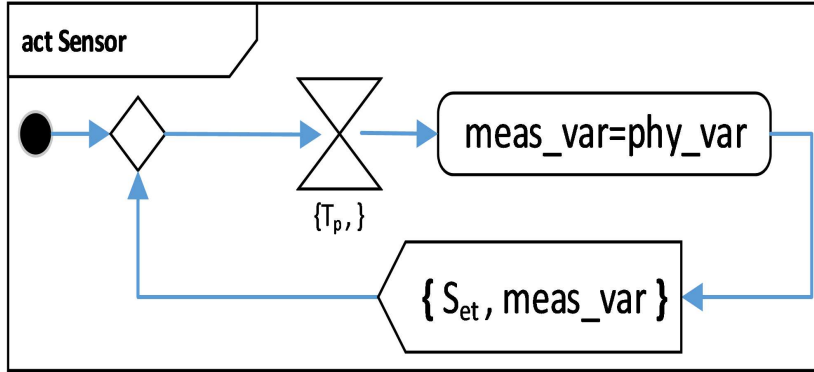


Fig. 4: SysML Activity Diagram of the Sensor.

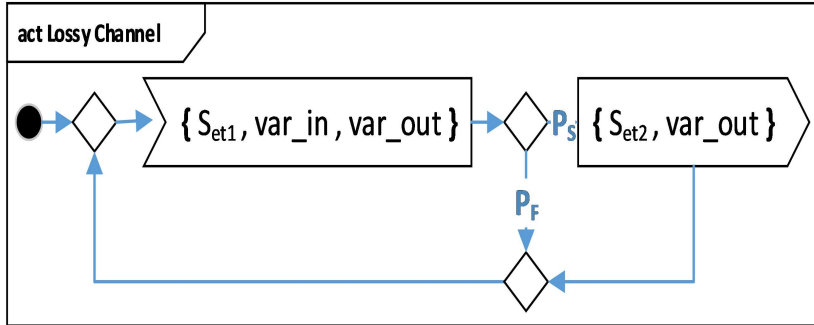


Fig. 5: SysML Activity Diagram of the Lossy Channel.

$$Act_Ctrl = l \mapsto l_1 : Mrg \mapsto N_1$$

$$N_1 = l_2 : Comp_Events(l_3 : \{S_{et1}, G, G_r\} ? \mapsto N_2, l_4 : D_{TB}(T_p,) \mapsto N_3)$$

$$N_2 = l_5 : CALL_P(IIR = Act_Calc_IIR(t)) \mapsto l_6 : Mrg \mapsto N_4$$

$$N_3 = l_7 : CALL_P(IIR = Act_Calc_IIR_missing(t)) \mapsto l_6$$

$$N_4 = l_8 : \{S_{et2}, IIR\} ! \mapsto l_1$$

$$Act_Actuator = l \mapsto l_1 : Mrg \mapsto l_2 : \{S_{et}, IIR_c, IIR_r\} ? \mapsto l_3 : ACT(IIR = IIR_r) \mapsto l_1$$

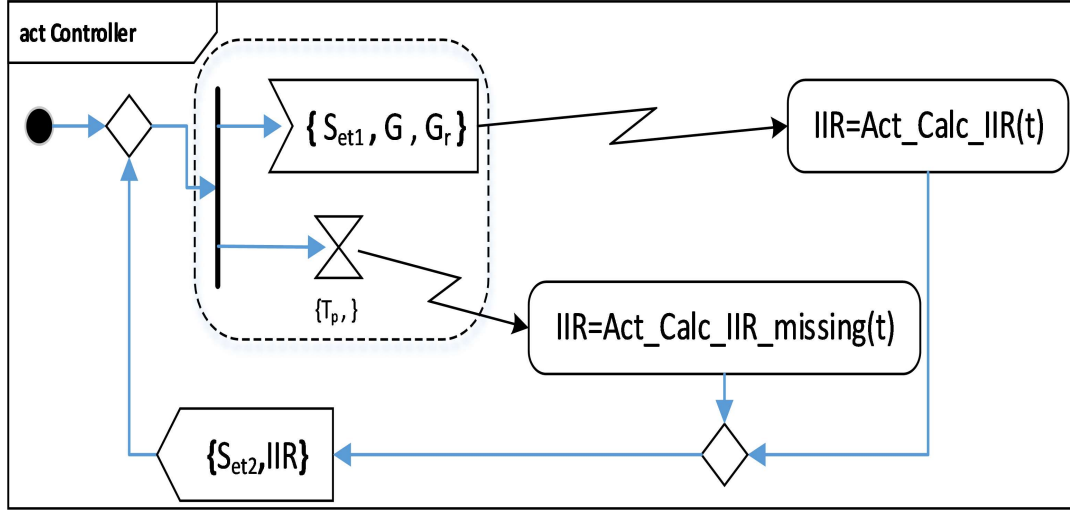


Fig. 6: SysML Activity Diagram of the Controller.

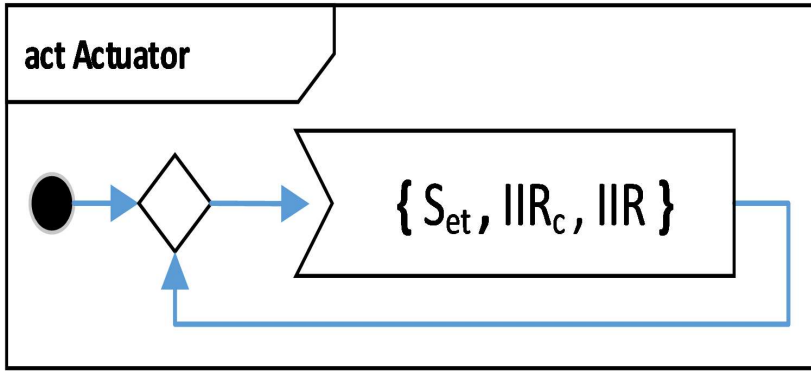


Fig. 7: SysML Activity Diagram of the Actuator.

$$Act_meal_scenario = l \mapsto l_1 : Mrg \mapsto l_2 : D_{TB}(inter_meal_time,) \mapsto N_1$$

$$N_1 = l_3 : ACT(cur_meal = 1000 * meal_carbs/meal_dur, D_meal = (Q_{sto1} + Q_{sto2})/1000) \mapsto N_2$$

$$N_2 = l_4 : D_{TB}(meal_dur, D_meal' == cur_meal/1000) \mapsto l_5 : ACT(cur_meal = 0) \mapsto l_1$$

❖ CPS architecture and flow for artificial pancreas

The SysML block definition diagram shown in Fig. 9 defines the blocks and their input/output ports. Also, the mapping of the blocks and the variables

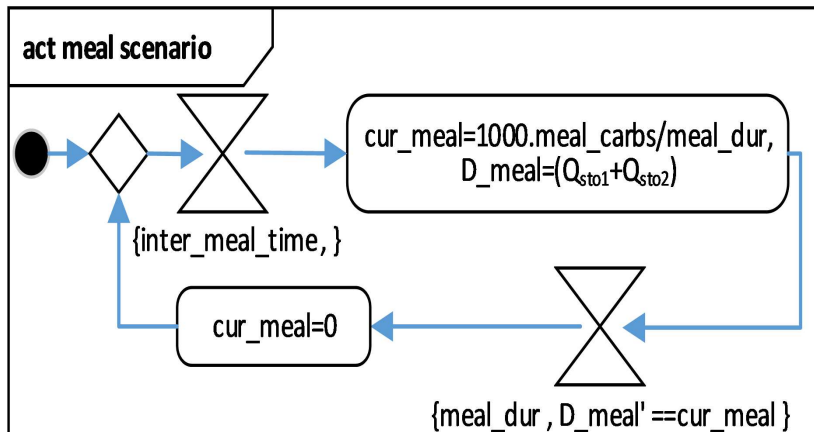


Fig. 8: SysML Activity Diagram of the Meal Scenario.

as well as the flow of information among these blocks are defined in the flow
internal block diagram shown in Fig. 10.

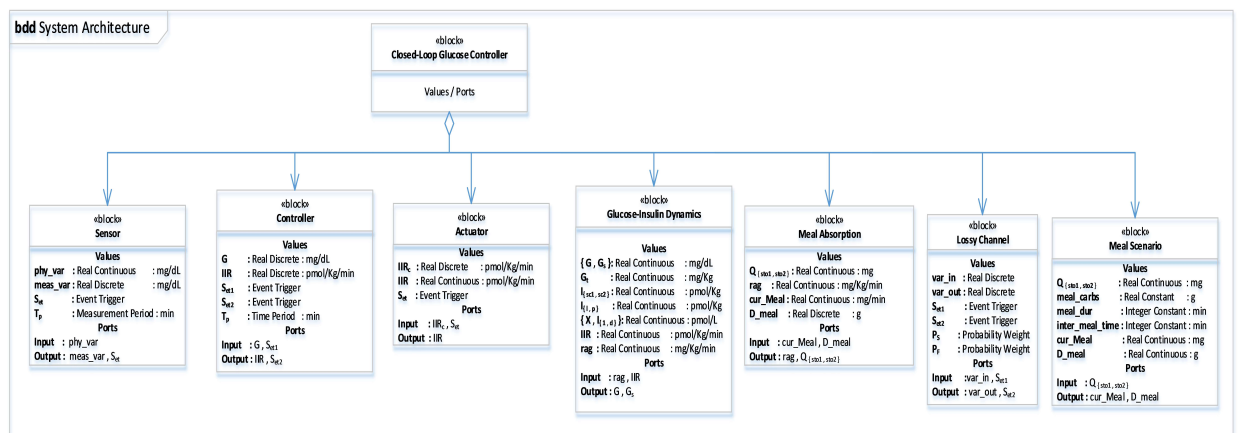


Fig. 9: SysML Architectural Block Definition Diagram of the Closed-Loop Glucose Control System.

4 CPS Semantics

The system behavior should be represented in the suitable formality that matches the language of the analysis tool. To do so, the SysML components are converted into a network of equivalent PTA models. In the following, the PTA is defined and the new proposed automated conversion procedure is presented.

Definition 2 (PTA). A PTA for CPS is a tuple $(L, l_0, L_{lbl}, L_{IP}, L_{OP}, E, X, V_g, INV(X, VAR), A(V_g), G(X, V_g), S_{et}, P_r)$, where:

- L is a finite set of locations,
- $l_0 \in L$ is the initial location,
- L_{lbl} is a set of labels,
- L_{IP} is a finite set of input ports,
- L_{OP} is a finite set of output ports,
- E is a finite set of edges,
- X is a finite set of clocks,
- VAR is a finite set of general-type variables,
- $INV(X, V_g)$ is a finite set of invariants over PTA clocks X and variables V_g , $A(V_g)$ is a finite set of actions on the variables V_g ,
- $G(X, V_g)$ is a finite set of atomic propositions on PTA clocks X and variables V_g , S_{et} is a finite set of synchronization event triggers, and
- P_r is a finite set of probabilistic weights.

Definition 3 (Semantics of CPS). Let $(L, l_0, L_{lbl}, L_{IP}, L_{OP}, E, X, V_g, INV(X, VAR), A(V_g), G(X, V_g), S_{et}, P_r)$ be a PTA for CPS. The semantics are defined as a hybrid transition system composed of a set of locations L interconnected by a set of edges E through sets of input ports IP and output ports OP , where:

- Locations $L = \{l_1, l_2, \dots, l_{n1}\}$, where the i^{th} location $l_i \in L$ labelled $label_i \in L_{lbl}$ having the invariant constraints $inv_i \in INV$ and connected to the input port x_{ip} and the output ports X_{op} is referred as $l_i(label_i, inv_i, x_{ip}, X_{op})$.
- Edges $E = \{e_1, e_2, \dots, e_{n2}\}$, where the i^{th} edge running the action $a \in A$ and triggering the synchronization event $s_{et} \in S_{et}$, and connected to the output port x_{op} and input port x_{ip} is referred as $e_i = \{a, s_{et}, x_{op}, x_{ip}\}$.
- Input ports $L_{IP} = \{l_{ip1}, l_{ip2}, \dots, l_{ip_{n1}}\}$, where the i^{th} input port $l_{ip_i} \in L_{IP}$ sourcing from incoming edges X_e towards the i^{th} location $l_i \in L$ and applying the action $a \in A$ is defined as $l_{ip_i} = \{a, X_e, i\}$.
- Output ports $L_{OP} = \{l_{op1}, l_{op2}, \dots, l_{op_{n3}}\}$, where the k^{th} output port $l_{op_k} \in L_{OP}$ sourcing from the i^{th} location l_i towards the j^{th} edge e_j , guarded by the atomic proposition $g \in G$, triggered by the event trigger $s_{et} \in S_{et}$, and having the probabilistic weight $p_r \in P_r$ is defined as $l_{op_k} = \{g, s_{et}, p_r, i, j\}$.

PTAs traverse sequentially through output ports towards edges, followed by input ports towards the next location, starting at an initial location denoted by l_0 . In the case of the PTA being at location l_i , the invariant inv_i must be satisfied as long as the PTA is at location l_i . Similarly, an output port that has a guard g with respect to its traversal can only be traversed if this guard g has

been satisfied. An output port with an event trigger s_{et} is synchronized with another PTA, so that the output port is only traversed when it is activated by the corresponding event trigger on the edge of the other PTA. Furthermore, an output port can be traversed among other output ports in a probabilistic manner by assigning a probability weight p_r to each of the possible candidates for traversal of the output port.

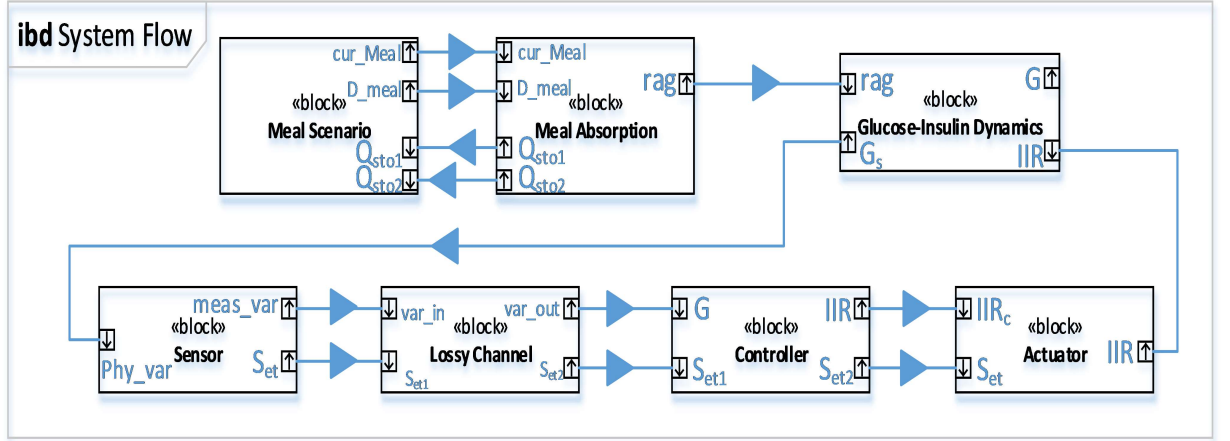


Fig. 10: SysML Flow Internal Block Diagram of the Closed-Loop Glucose Control System.

4.1 Converting SysML into Equivalent PTA

In order to analyze the CPS described in SysML, it is necessary to model the hybrid system in PTA. So, SysML blocks are translated into equivalent PTA blocks which are parallel-composed to construct the hybrid system's global behavior. The synchronization of actions and the transfer of values are specified using shared variables.

The template of each PTA is instantiated with its input/output parameters properly defined. The SysML flow internal block diagram (as in Fig. 10) is consulted to define global variables for the parameters connecting the PTA components of the system. When instantiating a PTA template, the parameters are passed by-reference except for constant parameters that are passed by-value. Instead, those constants can be defined as local variables in the PTA. The following rules govern the definition of variables in PTA models.

- Continuous real-valued parameters are defined using clock variables.

- Discrete real-valued parameters are defined using floating point variables.
- An event trigger should be activated whenever a discrete variable is updated, so that the other PTAs are notified about the new update.
- Discrete integer parameters are defined as integer variables and are passed between PTAs similar to the floating point variables.
- When assigning or initializing a numerical variable, it can be evaluated to a single value or to a range of values for a uniformly-distributed random assignment.

❖ Converting SysML EAC into PTA

This part presents the detailed procedure for constructing a PTA block that represents a SysML EAC block. Alongside the description of the conversion steps, an illustrative example is provided for converting the *Act_Channel_{lossy}* block from EAC into PTA.

- The first step is to merge all the EAC nodes into the main EAC construct. This is done by iterating through the auxiliary constructs (N_x) and substituting for them in the main construct as depicted in Fig. 11.

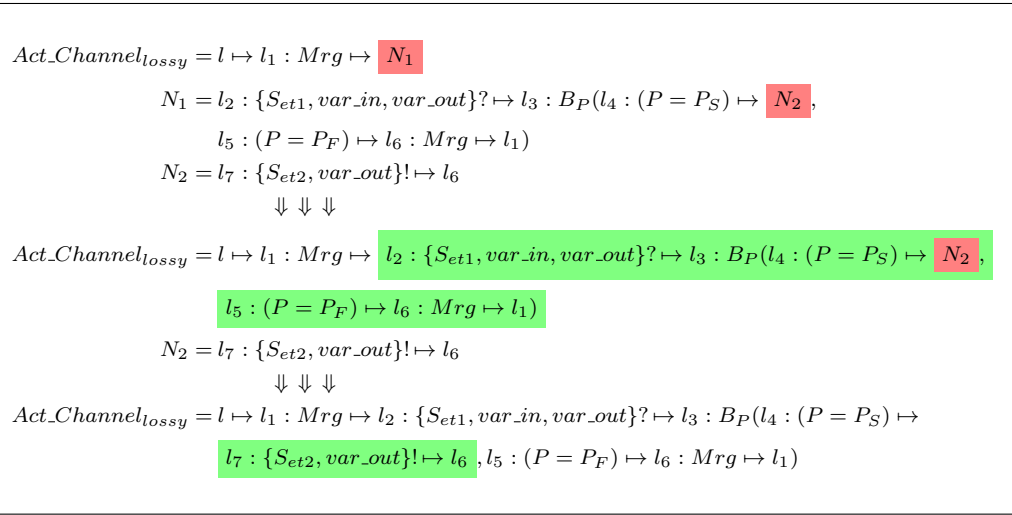


Fig. 11: EAC Lossy Channel Example - Merging Nodes

- Connecting the EAC terms so that each arrow is uniquely identified as presented in Fig. 12.
- Handling branching terms (B_P or *Comp_Events*) and replicating the EAC construct, so that each branching term has only one path at a time. This is done by iterating through the branching terms and taking one branch at a time as shown in Fig. 13.
- Building the PTA skeleton using the procedure described in Algorithm 1. The resulting skeleton for *Act_Channel_{lossy}* example is shown in Fig. 14.

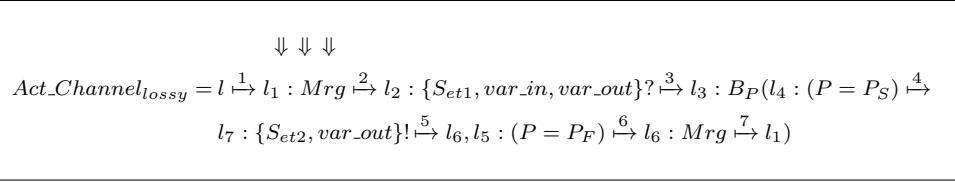


Fig. 12: EAC Lossy Channel Example - Labeling Arrows

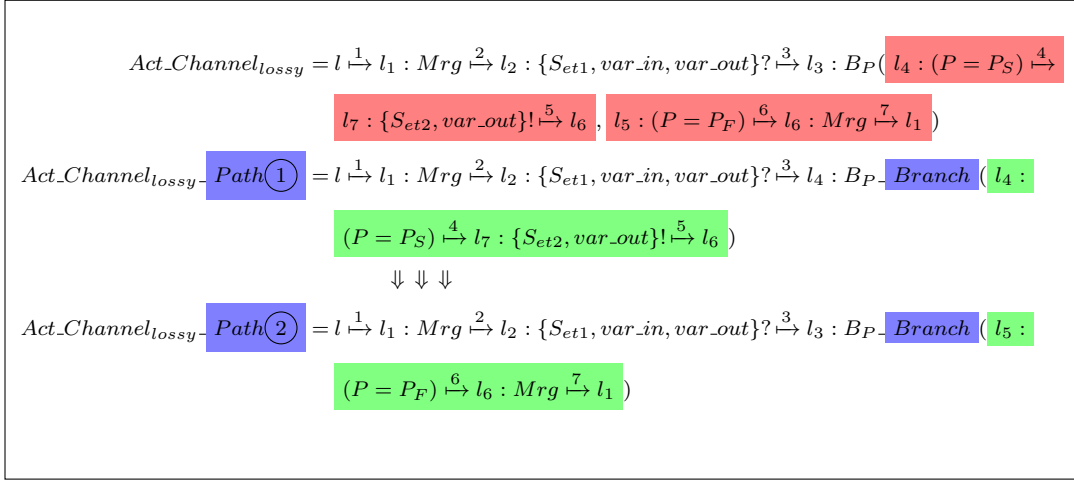


Fig. 13: EAC Lossy Channel Example - Branches Handling

- For each location node that has non-empty *prev* field, insert an input port.
- For locations with *next* field, insert an output port per edge node that is outgoing from the location. In the following steps, when an EAC term is linked to an output port, the one that is connected to the location where the EAC belongs is identified. In case the location is attached to two or more output ports, the sequence of EAC terms in the path construct is used to identify the corresponding output port. Moreover, an EAC node that shows up in more than one path is only converted once at its first appearance.
- Replacing the following EAC terms with their equivalent PTA terms.
 - EAC term l signifies the location as an initial location.
 - $D_{TB}(\tau_{min} : \tau_{max}, C)$: Declare a clock variable t , Add a reset for the clock ($t = 0$) to the input port action, Add the following constraint ($t \leq \tau_{max} \ \&\& \ C$) to the invariants *inv* of the location, and add the following ($t \geq \tau_{min}$) to the guard g of the output port.
 - $\{S, X_{src}, X_{dst}\}?$: Add the event trigger $S?$ to the respective field s_{et} of the output port, and add the assignment ($X_{dst} = X_{src}$) to the action of the edge outgoing from the output port.

Algorithm 1 Construction of PTA Skeleton.

```

for each:  $EAC\_Path$ 
1:  $prev\_Node = \emptyset$  ▷ The first node of a path has no predecessor.
for each:  $EAC\_Node \in EAC\_Path$ 

2: if  $EAC\_Node \in \{Mrg, Comp\_Events, B_P, D(*), \{*, *\}?, (P = *)\}$  then
3:    $EAC\_Type = LOCATION$ 
4: else if  $EAC\_Node \in \{\mapsto, ACT, CALL_P, \{*, *\}!\}$  then
5:    $EAC\_Type = EDGE$ 
6: end if

7: if  $EAC\_Node$  processed before then
8:    $cur\_Node = PTA\_Node[EAC\_Node]$  ▷ Traverse through the node.
9:    $cur\_Node.prev.addMember(prev\_Node)$  ▷ Create a new input port for the node.
10:   $prev\_Node.next.addMember(cur\_Node)$  ▷ Create a new output port for the node.
11: else if  $EAC\_Type == prev\_Node.type$  then
12:   $cur\_Node.EAC.addMember(EAC\_Node)$  ▷ A compliment for the previous node.
13: else ▷ A node not processed yet.
14:   $cur\_Node = create\_Node(type = EAC\_Type)$  ▷ Create the node.
15:   $cur\_Node.EAC.addMember(EAC\_Node)$  ▷ Traverse through the node.
16:   $cur\_Node.prev.addMember(prev\_Node)$  ▷ Create an input port.
17:   $prev\_Node.next.addMember(cur\_Node)$  ▷ Create an output port.
18: end if

```

$Node_{ID} = [$	$type,$	$prev,$	$next,$	EAC	$]$
$Node_1 = [$	$LOCATION,$	$\emptyset,$	$2,$	l	$]$
$Node_2 = [$	$EDGE,$	$1,$	$3,$	$\xrightarrow{1}$	$]$
$Node_3 = [$	$LOCATION,$	$\{2, 11\},$	$4,$	l_1	$]$
$Node_4 = [$	$EDGE,$	$3,$	$5,$	$\xrightarrow{2}$	$]$
$Node_5 = [$	$LOCATION,$	$4,$	$6,$	l_2	$]$
$Node_6 = [$	$EDGE,$	$5,$	$7,$	$\xrightarrow{3}$	$]$
$Node_7 = [$	$LOCATION,$	$6,$	$\{8, 10\},$	$\{l_3, l_4, l_5\}$	$]$
$Node_8 = [$	$EDGE,$	$7,$	$9,$	$\{\xrightarrow{4}, l_7, \xrightarrow{5}\}$	$]$
$Node_9 = [$	$LOCATION,$	$\{8, 10\},$	$11,$	l_6	$]$
$Node_{10} = [$	$EDGE,$	$7,$	$9,$	$\xrightarrow{6}$	$]$
$Node_{11} = [$	$EDGE,$	$9,$	$3,$	$\xrightarrow{7}$	$]$

Fig. 14: EAC Lossy Channel Example - Building Skeleton.

- $\{S, X\}!$: Add this event trigger $S!$ to the respective event trigger field s_{et} of the containing edge.
 - $(P = p_x)$: Add the following probabilistic weight to the corresponding field p_r of the output port.
 - $ACT(A)$: Add the action A to the corresponding field a of the edge.
 - $CALL_P(A)$: Add the behavior call $A()$ to the action field a of the edge.
- The results shown in Fig. 15 are obtained when applying the above rules on the $Act_Channel_{lossy}$ example:

```

loc1(label1,  $\phi$ ,  $\phi$ ,  $op_1$ )
loc2(label2,  $\phi$ ,  $ip_2$ ,  $op_2$ )
loc3(label3,  $\phi$ ,  $ip_3$ ,  $op_3$ )
loc4(label4,  $\phi$ ,  $ip_4$ ,  $\{op_{(4,1)}, op_{(4,2)}\}$ )
loc5(label5,  $\phi$ ,  $ip_5$ ,  $op_5$ )
ip2( $\phi$ ,  $\{e_1, e_6\}$ , loc2)
ip3( $\phi$ ,  $e_2$ , loc3)
ip4( $\phi$ ,  $e_3$ , loc4)
ip5( $\phi$ ,  $\{e_4, e_5\}$ , loc5)
op1( $\phi$ ,  $\phi$ ,  $\phi$ , loc1,  $e_1$ )
op2( $\phi$ ,  $\phi$ ,  $\phi$ , loc2,  $e_2$ )
op3( $\phi$ ,  $Set_1?$ ,  $\phi$ , loc3,  $e_3$ )
op(4,1)( $\phi$ ,  $\phi$ ,  $P_S$ , loc4,  $e_4$ )
op(4,2)( $\phi$ ,  $\phi$ ,  $P_F$ , loc4,  $e_5$ )
op5( $\phi$ ,  $\phi$ ,  $\phi$ , loc5,  $e_6$ )
e1( $\phi$ ,  $\phi$ ,  $op_1$ ,  $ip_2$ )
e2( $\phi$ ,  $\phi$ ,  $op_2$ ,  $ip_3$ )
e3( $var_{out} = var_{in}$ ,  $\phi$ ,  $op_3$ ,  $ip_4$ )
e4( $\phi$ ,  $Set_2!$ ,  $op_{(4,1)}$ ,  $ip_5$ )
e5( $\phi$ ,  $\phi$ ,  $op_{(4,2)}$ ,  $ip_5$ )
e6( $\phi$ ,  $\phi$ ,  $op_5$ ,  $ip_2$ )

```

Fig. 15: EAC Lossy Channel Example - Replacing EAC with PTA Terms.

- After each EAC receive node, insert a new location between the event trigger and the signal sampling. Also, a new location is added when an output port with a probabilistic weight is directly followed by an edge with an EAC send node. This is done so that the send node is separated from the output port. When applying this on the $Act_Channel_{lossy}$, the results look like Fig. 16

loc_1	$(label_1, \phi, \phi, op_1)$
loc_2	$(label_2, \phi, ip_2, op_2)$
loc_3	$(label_3, \phi, ip_3, op_3)$
loc_4	$(label_4, \phi, ip_4, \{op_{(4,1)}, op_{(4,2)}\})$
loc_5	$(label_5, \phi, ip_5, op_5)$
loc_6	$(label_6, \phi, ip_6, op_6)$
loc_7	$(label_7, \phi, ip_7, op_7)$
ip_2	$(\phi, \{e_1, e_6\}, loc_2)$
ip_3	(ϕ, e_2, loc_3)
ip_4	(ϕ, e_3, loc_4)
ip_5	$(\phi, \{e_8, e_5\}, loc_5)$
ip_6	(ϕ, e_7, loc_6)
ip_7	(ϕ, e_4, loc_7)
op_1	$(\phi, \phi, \phi, loc_1, e_1)$
op_2	$(\phi, \phi, \phi, loc_2, e_2)$
op_3	$(\phi, Set_1?, \phi, loc_3, e_7)$
$op_{(4,1)}$	$(\phi, \phi, P_S, loc_4, e_4)$
$op_{(4,2)}$	$(\phi, \phi, P_F, loc_4, e_5)$
op_5	$(\phi, \phi, \phi, loc_5, e_6)$
op_6	$(\phi, \phi, \phi, loc_6, e_3)$
op_7	$(\phi, \phi, \phi, loc_7, e_8)$
e_1	(ϕ, ϕ, op_1, ip_2)
e_2	(ϕ, ϕ, op_2, ip_3)
e_3	$(var_{out} = var_{in}, \phi, op_6, ip_4)$
e_4	$(\phi, \phi, op_{(4,1)}, ip_7)$
e_5	$(\phi, \phi, op_{(4,2)}, ip_5)$
e_6	(ϕ, ϕ, op_5, ip_2)
e_7	(ϕ, ϕ, op_3, ip_6)
e_8	$(\phi, Set_2!, op_7, ip_5)$

Fig. 16: EAC Lossy Channel Example - Inserting Locations

- Divide the locations into transient and regular (time-consuming) locations. A regular location is identified by having either a guard or an event trigger on the output port, or by having a non-empty invariant field. For the $Act_Channel_{lossy}$ example, all the locations are transient except location loc_3 which has an event trigger on the output port.

- The rate of all local clocks should be identified on all regular locations. Therefore, if a clock is not supposed to evolve in a specific regular location, its evolution rate should be assigned to 0 in the invariants field of that location.
- When exporting the PTAs into an XML file compatible with UPPAAL-SMC analyzer, transient locations are specified as *urgent* locations except for the following:
 - A location which emits output ports with probabilistic weights (location loc_4 in $Act_Channel_{lossy}$ example) is defined as an *anchor* point (for syntax compatibility).
 - The first location following a receive node (location loc_6 in $Act_Channel_{lossy}$ example) should be set to *committed* for synchronization correctness (semantic compatibility).

The resulting PTA diagram for the above transformed lossy channel is depicted in Fig. 17. This PTA initializes at the location loc_1 . This location is urgent which means that no time progress and hence the PTA will move instantly through the output port op_1 , the edge e_1 , the input port ip_2 to the next location loc_2 . This location is also an urgent location and hence the PTA will move through the output port op_2 , edge e_2 , and the input port ip_2 towards the location loc_3 . The output port op_3 is activated by the event trigger S_{et1} ? which is controlled by another PTA (the sensor in this case). Then, this sensor activates the event trigger S_{et} to send a new measurement (the variable var_{in}) through the wireless channel. When triggered by the event trigger S_{et1} , the lossy channel PTA moves through the output port op_3 , the edge e_7 , and the input port ip_6 towards the committed location loc_6 . Like the urgent location, a committed location freezes time but also synchronizes the PTAs so that the correct sequence of actions takes place. In this PTA, it is required so that the up-to-date version of the measurement value var_{in} is read.

The PTA moves through op_6 towards the edge e_3 where the measurement is sampled, and then through the input port ip_4 to the location loc_4 which is a probabilistic branching point. Then, the PTA will take a branch depending on probability weights. At one branch, the message will get lost and so the PTA takes the output port $op_{(4,2)}$ towards the edge e_5 and the input port ip_5 to reach the location loc_5 . In the other branch, the measurement is successfully relayed so the other PTA (the controller in this case) is notified with the event trigger $S_{et2}!$, so the PTA moves through $op_{(4,1)}$, e_4 , ip_7 to the transient location loc_7 towards the output port op_7 and the edge e_8 (where $S_{et2}!$ is activated) to the input port ip_5 while merging with the other branch in the location loc_5 . Finally, the PTA moves via the output port op_5 and the edge e_6 through the input port ip_2 to merge in the location loc_2 .

❖ modeling ODESCD using PTA

The same rules apply to convert ODESCD into PTA where the ODE variables X are defined as clock variables. The PTA is composed of one location where the rates of the ODE variables X are assigned using equality constraints in the invariant field of the main location. If some variables or parameters are

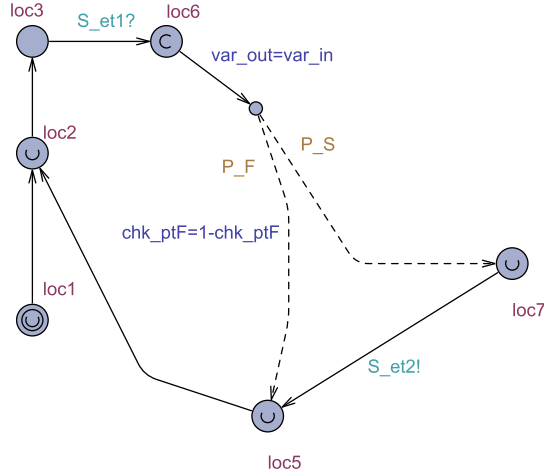


Fig. 17: The Resulting PTA Diagram for the Lossy Channel

initialized with random values, an additional transient initial location is added with the variables assigned in the edge connecting the initial location to the main operational location.

4.2 Soundness

After presenting the semantics of CPS and PTA, we prove the soundness of the developed framework. First, let Γ to be a function denoting Algorithm 1. Now, we prove the soundness of the transformation by showing that Γ guarantees the integrity of the CPS design, i.e. no added, modified, or excluded behavior. Thus, an equivalent PTA behavioral model is produced. Then, we show that the soundness proves the satisfiability preservation of MILT expressions when applying Γ .

As depicted in Fig. 18, we have to show the nature of the relation $\mathcal{R}$, that compares both PTA^{cps} and PTA^f constructed through EAC and PTA semantics rules respectively, while preserving both behaviours. Indeed, the relation $\mathcal{R}$ could be determined by comparing the semantics of each term in EAC and the semantics of its image obtained by the function Γ . Since the goal is to guarantee the behaviour integrity of PTA^{cps} and the resulting PTA^f should not differ from PTA^{cps} , Lemma 1 proves that $\mathcal{R}$ is a bisimulation relation.

Lemma 1 *The binary relation $\mathcal{R}$, is a bisimulation, whenever $S\mathcal{R}\hat{S}$, satisfies the following.*

1. If $S \xrightarrow{\alpha} S'$ then $\exists \hat{S}'$ such that $\hat{S} \xrightarrow{\alpha} \hat{S}'$ and $S' \equiv_{\mathcal{R}} \hat{S}'$.

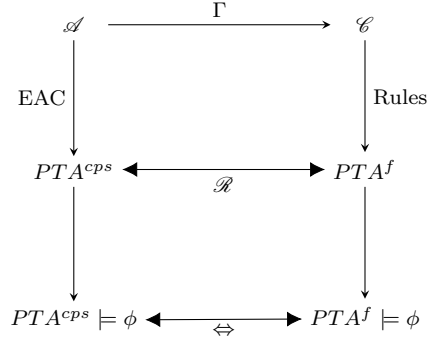


Fig. 18: The Transformation Soundness Schema.

2. If $\hat{S} \xrightarrow{\alpha} \hat{S}'$ then $\exists S$ such that $S' \xrightarrow{\alpha} S'$ and $\hat{S}' \equiv_{\mathcal{R}} S'$.

Proof Let's consider $A \in PTA^{cps}$ and $B \in PTA^f$ where $\Gamma(A) = B$. So, by induction on EAC terms, we prove that $\mathcal{R}$ is a bisimulation binary relation as follows.

- When $A = i \rightarrow \mathcal{N}$, then based on the rule $\exists S \xrightarrow{\alpha} S'$ such that $S = i \rightarrow \mathcal{N}$ and $S' = i \rightarrow \mathcal{N}$, we will have, $\Gamma(A) = \Gamma(i \rightarrow \mathcal{N}) = \text{initial to } i$. Thus, $\text{initial} \wedge \neg i \xrightarrow{\alpha} \neg \text{initial} \wedge i \in B^s$. Then, $PTA^{cps} \mathcal{R} PTA^f$ when $A = i \rightarrow \mathcal{N}$.
- For $\{S, X\}! \rightarrow \mathcal{N}$, then $\overline{X} \rightarrow \mathcal{N} \rightarrow \{S, X, X'\}! \rightarrow \mathcal{N} \in PTA^{cps}$. Also, $\Gamma(A) = \text{resource} < v > \rightarrow \mathcal{N}$ which means $\text{resource}_v \wedge \neg \mathcal{N} \xrightarrow{\text{prt}} \neg \text{resource}_v \wedge \mathcal{N} \in B^s$. So, $PTA^{cps} \mathcal{R} PTA^f$.
- In the case of $A = \text{resource?} v \rightarrow \mathcal{N}$, we have $\overline{\text{resource?} v} \rightarrow \mathcal{N} \rightarrow \text{resource?} v \rightarrow \mathcal{N} \in B^{sn}$. Thus, $PTA^{cps} \mathcal{R} PTA^f$.
- If $A = \text{resource!} v \rightarrow \mathcal{N}$, we have $\overline{\text{resource!} v} \rightarrow \mathcal{N} \rightarrow \text{resource!} v \rightarrow \mathcal{N} \in B^{sn}$. $R \text{ resourceout}_v \wedge \exists v \wedge \neg \mathcal{N} \xrightarrow{\text{prt}} \neg \text{resourceout}_v \wedge \mathcal{N} \in B^s$.
- By considering $A = \text{resource} \uparrow \text{expression} \rightarrow \mathcal{N}$, then $\overline{\text{resource} \uparrow \text{expression}} \rightarrow \mathcal{N} \rightarrow \text{resource} \uparrow \text{expression} \rightarrow \mathcal{N} \in B^{sn}$. As a result, we have $\text{resource}_v \wedge \neg \mathcal{N} \xrightarrow{\text{prt}} \neg \text{resource}_v \wedge v = \text{newvalue} \wedge \mathcal{N} \in B^s$, which means $PTA^{cps} \mathcal{R} PTA^f$.
- For the decision term $A = D(g_{v1}, \mathcal{N}_1, \mathcal{N}_2)$, we differentiate two cases:
 1. When $\neg g_{v1} \models \top$, we have $\overline{D(g_{v1}, \mathcal{N}_1, \mathcal{N}_2)} \xrightarrow{\neg g_{v1}} D(g_{v1}, \mathcal{N}_1, \mathcal{N}_2) \in B^{sn}$ by relying on the decision rule. Also, we have: $\Gamma(A) = \{\text{on prt}_i \text{ from source to } \mathcal{N} \text{ provided } g_{v_i} = \text{eval}(v_i) : i \in \{1, 2\}\}$. Also, since $\neg g_{v1} \models \top$, we have: $\overline{D(g_{v1}, \mathcal{N}_1, \mathcal{N}_2)} \xrightarrow{\neg g_{v1}} D(g_{v1}, \mathcal{N}_1, \mathcal{N}_2) \in B^s$.
 2. For the other case, when $g_{v1} \models \top$, we have shown that $D(g_{v1}, \mathcal{N}_1, \mathcal{N}_2) \equiv D(\neg g_{v1}, \mathcal{N}_2, \mathcal{N}_1)$. Thus, $PTA^{cps} \mathcal{R} PTA^f$.

We have shown that for each EAC term, we have $PTA^{cps} \mathcal{R} PTA^f$ in which result that $\mathcal{R}$, is a bisimulation relation and it is symmetric.

Based on the illustration presented in Fig 18, the transformation's objective is to verify functional properties of the generated PTA model and then

infer satisfiability results for the CPS design. Using Lemma 1, Proposition 1 demonstrates how the properties expressed in MITL logic can be satisfied.

Proposition 1 $\forall A \in PTA^{cps}, B \in PTA^f$ s.t. $\Gamma(A) = B$, we have: $\forall \phi \in MITL : PTA^f \models \phi \implies PTA^{cps} \models \phi$.

Proof By induction on MITL terms, we prove that $B \models \phi \implies A \models \phi$.

1. First, let's consider the state formulae $\phi = \varphi_1 \wedge \varphi_2$ where $B \models \phi$. Now, we show the satisfiability of ϕ on A for the following EAC terms.
 - For $A = i \rightarrow \mathcal{N}$, we have $i \rightarrow \mathcal{N} \xrightarrow{\alpha} \overline{i \rightarrow \mathcal{N}} R \text{ initial} \wedge \neg i \xrightarrow{\alpha} \neg \text{initial} \wedge i$. If $\text{initial} \wedge \neg i \models \varphi_1 \wedge \varphi_2$ means $\text{initial} \wedge \neg i = \varphi_1 \wedge \varphi_2$. Thus, $i \rightarrow \mathcal{N} \models \phi$, and, $B \models \phi$
 - For $A = \text{resource} < v > \rightarrow \mathcal{N}$ when $\neg \text{resource}_v \wedge \mathcal{N} \models \varphi_1 \wedge \varphi_2$, we have $\text{resource} < v > \rightarrow \mathcal{N} \models \varphi_1 \wedge \varphi_2$. Then, $B \models \phi$.
 - For $A = \text{resource}?v \rightarrow \mathcal{N}$, then $B \models \phi$ $\text{resourcein}_v \wedge v = \text{newvalue} \wedge \neg \mathcal{N} \xrightarrow{prt} \neg \text{resourcein}_v \wedge \mathcal{N} \models \phi$. Thus, we have $\overline{\text{resource}?v \rightarrow \mathcal{N}} \xrightarrow{\alpha} \text{resource}?v \rightarrow \mathcal{N} \models \phi$. Consequently, $B \models \phi$.
2. Now, we consider the path formulae $P_{\wp p}[\psi]$. So, since EAC does not support probabilistic decisions and has only deterministic ones, $P_{\geq 1}[\psi]$ means ψ else we consider the case of $P_{\leq 0}[\psi]$. Then, we prove by induction on the path operators that $PTA^{cps} \models \phi$ when $PTA^f \models \phi$ as follows.
 - For $\phi = N\varphi$, $B \models \phi$ means $\exists \hat{S} \xrightarrow{\alpha} \hat{S}' \in B^n$ such that $\hat{S}' \models \varphi$. In addition, since $\mathcal{R}$ is symmetric, then $\exists S \xrightarrow{\alpha} S' \in B$ such that: $S' \models \varphi$.
 - For $\phi = \varphi_1 \cup^t \varphi_2$, we have $\exists \hat{S}_1 \xrightarrow{\alpha} \dots \rightarrow \hat{S}_t \subseteq B^n$ such that $\hat{S}_{i:i < t'} \models \varphi_1$ and $\hat{S}_2 \models \varphi_2$. Also, $\mathcal{R}$ is symmetric and $\exists S_1 \xrightarrow{\alpha} \dots \rightarrow S'_t \subseteq B^{sn}$ where $S_i \mathcal{R} \hat{S}_i : 0 < i \leq t$. Thus, $B \models \phi$.

Based on the previous proof, we have shown that for each EAC and MITL term, $\mathcal{R}$ always preserves the satisfiability of MITL formulae. Consequently, $B \models \phi \implies A \models \phi$ for all ϕ expressed in MITL when $PTA^{cps} \mathcal{R} PTA^f$.

5 Experimentation

This section shows the effectiveness of the proposed framework by first validating the transformation algorithm. Then, the proposed approach is used to demonstrate how the safety of the obtained model can be examined by statistical model checking over a list of selected functional and safety requirements.

5.1 Validation of the Conversion Procedure

In order to demonstrate the correctness of the proposed approach, PTA models are validated. Properties are specified for each component of the system that constrain its functional behavior. To evaluate whether the resulting PTA model meets the behavioral properties, random simulations are conducted and trace

log analysis is applied to the results. The resulting PTA models are more likely to be valid representations of the CPS components when all the properties are satisfied.

By comparing the values of the ODE variables with a mathematical ODE solver, PTAs representing ODESCD are validated. In the case of the ODE-SCDs describing meal absorption and glucose-insulin dynamics, multiple simulations are conducted on 10 virtual patients for 24 hours under various meal scenarios. The PTAs for these ODESCDs that are constructed using the above automatic procedure are simulated.

The trace logs of the physical variables are compared against our ODE solver developed in Matlab and errors are recorded. The absolute errors of variable samples are divided by the variable root mean square to get the relative absolute errors. The percentage mean and standard deviation (std) of these relative absolute errors are depicted in Table 2. It can be noted that the relative errors are negligible and hence demonstrate the correctness of the proposed procedure.

Table 2: Meal and Glucose-Insulin Dynamics ODESCD Variables (Results Against a Mathematical Solver).

Variable Identifier	Relative Absolute Error {mean+std}
Q_{sto1}	$0.018\% \pm 0.007\%$
Q_{sto2}	$0.027\% \pm 0.012\%$
rag	$0.028\% \pm 0.012\%$
I_{sc1}	$0.166\% \pm 0.049\%$
I_{sc2}	$0.117\% \pm 0.039\%$
X_1	$0.219\% \pm 0.029\%$
G_s	$0.164\% \pm 0.203\%$
I_1	$0.071\% \pm 0.030\%$
I_d	$0.047\% \pm 0.027\%$
I_l	$0.118\% \pm 0.040\%$
I_p	$0.118\% \pm 0.040\%$
G	$0.165\% \pm 0.200\%$
G_s	$0.180\% \pm 0.209\%$

For the case of cyber components which are specified by EAC, the following steps demonstrate the model validation for this type of PTAs.

- Sensor: The sensor PTA shown in Fig. 19-a has three locations. It periodically waits in loc_3 before sampling the subcutaneous glucose measurement phy_var into the variable $meas_var$. The edge originating from loc_3 to loc_2 synchronizes the sensor with the lossy channel by means of the event trigger S_{et} .
- A new measurement is sent periodically every T_p minutes: to check on this property, a new binary flag variable is added to the PTA (chk_pt_1 in the sensor PTA shown in the graph of Fig. 19-a). The variable is marked

whenever a measurement is sent. This can be achieved by flipping the value of the variable in an ACT term **at** the same edge as the send term (the edge goes from *loc3* to *loc2*). The variable is monitored on random simulations and its value should be flipped periodically every T_p minute.

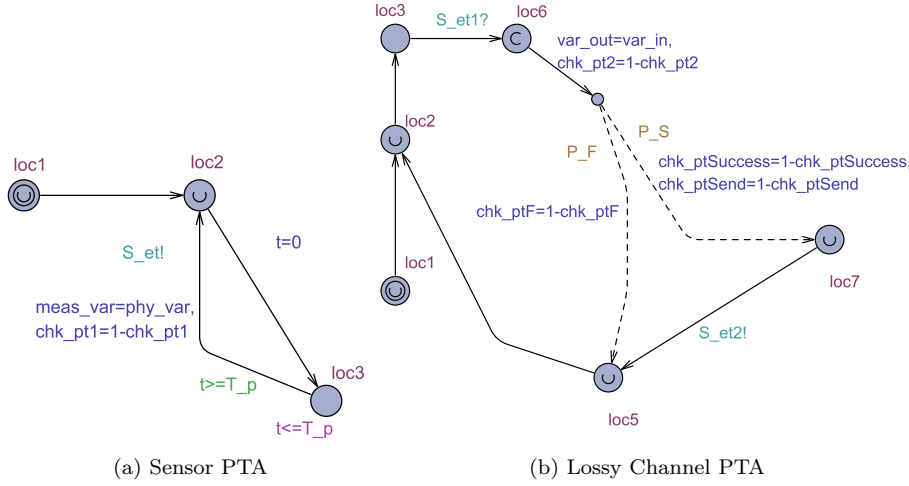


Fig. 19: A Part of the Sensor's PTA Communication Network.

- Whenever a measurement is sent, its value should be equal to the most recent sample of the physical variable monitored. **Then**, the value of the measurement is examined in particular whenever the binary flag, defined above, is flipped.
- The mapping of all the variables that are shared with other PTAs should be validated as well. In particular, the variables (*phy_var*, *S_{et}*, *meas_var*) in the Sensor PTA are examined against G_s in the glucose-insulin dynamics PTA and (*S_{et1}*, *var_in*) in the *Act_Channel_{lossy}* PTA, respectively. For a properly mapped system, the values of the variables in a PTA should be matched to their corresponding ones in all other PTAs at any time.
- *Channel_{lossy}*: The PTA shown in Fig. 19-b has seven locations where the edge from *loc3* towards *loc6* synchronizes with the sensor PTA to receive the measurement value as an input variable *var_{in}*. Similarly, the edge from *loc7* to *loc5* synchronizes with the controller PTA to send the measurement value as an output variable *var_{out}*.
- For every received measurement, the PTA will either successfully relay the measurement to the controller with probability P_S or fail with prob-

ability P_F . To check on this, binary flags are marked (flipped) on the corresponding edges for success and failure ($chk_pt_{Success}$ and chk_pt_F in the graph of Fig. 19-b). These binary flags are monitored for random simulations over various probabilistic weights.

- A measurement is sent to the controller if and only if the edge with P_S probabilistic weight is traversed. This can be checked by examining the corresponding binary flags.
- Whenever a measurement is sent to the controller (S_{et2} is activated), the value of the measurement (var_out) should be equal to the value of the sample received from the sensor (var_in).
- To validate the mapping of variables, the values of the variables (S_{et2} , var_out) should be equal to the values of the corresponding variables in the controller PTA (S_{et1} , G), respectively.
- Controller: The PTA shown in Fig. 20-a has five locations where the edge from loc_3 towards loc_5 synchronizes with the lossy channel PTA to receive the glucose measurement value as an input variable G . Similarly, the edge from loc_4 to loc_2 synchronizes with the actuator PTA to send the control value as an output variable IIR .

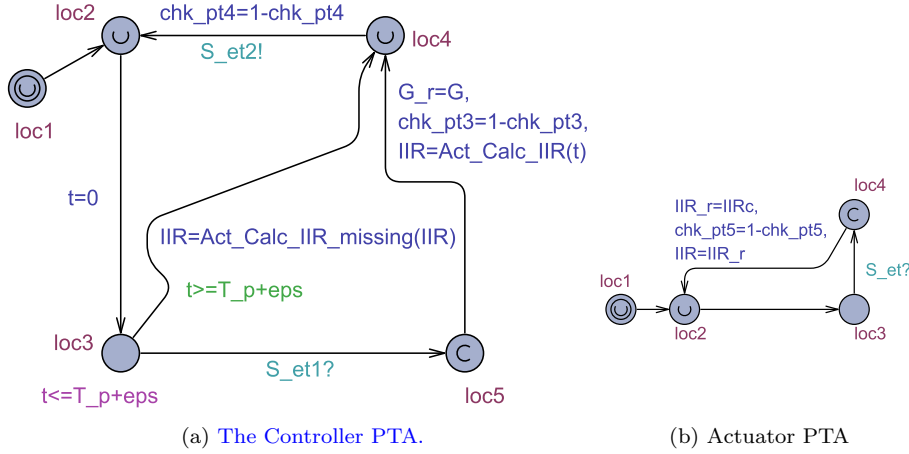


Fig. 20: A Part of the Artificial Pancreas Control Network.

- For each measurement delivered (S_{et1} activated), the PTA will read the measurement value G and use it to calculate a new Insulin Infusion Rate (IIR) using the standard [Proportional-Integral-Derivative \(PID\)](#) control [4, 34]. This new calculated value of IIR should be sent to the actuator by activating the event trigger S_{et2} .
- If the time since the last delivered measurement exceeds the control period T_p , the value of the variable IIR is zeroed and the event trigger S_{et2} is activated to command insulin delivery suspension.

- To validate the mapping of variables, the values of the variables (S_{et2} , IIR) should be equal to the values of the corresponding variables in the actuator PTA (S_{et} , IIR_c), respectively.
- Actuator: The PTA shown in Fig. 20-b has four locations where the edge from loc_3 towards loc_4 synchronizes with the controller PTA to receive the control value as an input variable IIR_c . The actuator then modifies the corresponding physical values in the glucose-insulin dynamics PTA through the output variable IIR .
 - Whenever a new infusion rate value IIR_c control command from the controller PTA is received (S_{et} activation), the actuator should update the value of the physical real-time variable IIR .
 - To verify the mapping of variables, the values for the variables IIR in both PTAs, actuator and glucose-insulin dynamics, should be equal at all times.
- Meal Scenario: This PTA is used to assign the input variables of the meal absorption model such as the carbohydrate amounts and the inter-meal times.
 - Each of the variables ($meal_carbs$, $meal_dur$, $inter_meal_time$) takes a value ranging between the configured minimum and maximum with uniform distribution. Based on the histogram of the variables, this can be validated.
 - The PTA should generate the values of the real-time variables (cur_meal , D_{meal}) complying with the right amounts of insulin-carbs, meal durations, and inter-meal times.
 - Validation for the mapping of the variables (cur_meal , D_{meal} , Q_{sto1} , Q_{sto2}) with their corresponding variables in the meal absorption PTA.
- Meal Absorption & Glucose-Insulin Dynamics:
 - The variables of the ODEs for both PTAs are observed and compared using our ODE simulator. The values for all variables should be identical to the ones calculated by the mathematical ODE solver developed in Matlab except for marginal numerical computational errors, e.g. precision.

5.2 Model Verification

PTAs are constructed for all the CPS components and are exported to a file for verification and analysis. This file is loaded into UPPAAL-SMC. A network of PTAs is created by instantiating and parallel-composing the PTA blocks using the UPPAAL-SMC. The tool performs hypothesis testing on queries specified by Metric Interval Temporal Logic (MITL). Also, monitor-based verification [8] could be used to specify more complicated queries using simpler expressions or for queries that are beyond the expressive power of MITL query language.

To demonstrate the use of the proposed framework to analyze real-life systems, UPPAAL-SMC is utilized to investigate safety properties of the artificial pancreas CPS that is supposed to regulate the blood glucose levels using

a pre-configured closed-loop control strategy. A good control strategy would be able to satisfy safety properties under normal conditions. Moreover, it would accommodate disturbances and minimize the side effects of faults.

Using this system, the sensor periodically transmits measurements to the controller over a wireless channel, but wireless packet transmission failure can cause measurements to be missing. Missing measurements can be handled using different control approaches. With the proposed SMC modeling and analysis, it is possible to evaluate whether each control approach can preserve safety properties at various error rates.

Whenever the controller receives a measurement, it calculates the required insulin rate using the standard PID. For a missing measurement, the controller will behave in one of three ways.

- Sustain: The controller will keep configuring the last valid calculated insulin rate until a new valid measurement is received.
- Suspend: The controller will stop insulin delivery until a new valid measurement is received.
- Revert: The controller will revert to a low value which is equal to the PID controller basal insulin rate until a new valid measurement is received.

The analysis is conducted on a database of 10 adult patients publicly accessible [38]. Each patient receives random meals of (20–50) *grams* carbohydrates each. Per patient, the analysis evaluates whether or not the controller satisfies safety properties for each of the three control configurations: sustain, suspend or revert. The following two safety properties are defined for analysis.

- S_A : At all times, the blood glucose levels should not cross the boundaries of severe minimum and maximum values of 50 *mg/dL* and 300 *mg/dL*, respectively.
- S_B : Whenever the glucose elevates to values higher than the threshold of 180 *mg/dL*, it should restore its value to normal range below this threshold within a maximum of two and a half hours.

The first safety property S_A is straightforward and can be described using the following MITL query:

$$\Pr[t \leq 1440] \left(\Box G \geq 50 \ \&\& \ G \leq 300 \right) \geq 0.99$$

This property specifies that throughout the test duration of one day (1440 minutes) the blood glucose levels should be limited between 50 *mg/dL* and 300 *mg/dL* with a probability above or equal 99%. On the other side, the second safety property S_B is too elaborate to describe in a query using MITL. Instead, a monitor PTA is designed to observe the time duration for each time the glucose level elevates above 180 *mg/dL* as shown in Fig. 21. Having this variable (tg_{180}) assigned, the safety property S_B is described using the following MITL property.

$$\Pr[t \leq 1440] \left(\Box tg_{180} \leq 150 \right) \geq 0.99$$

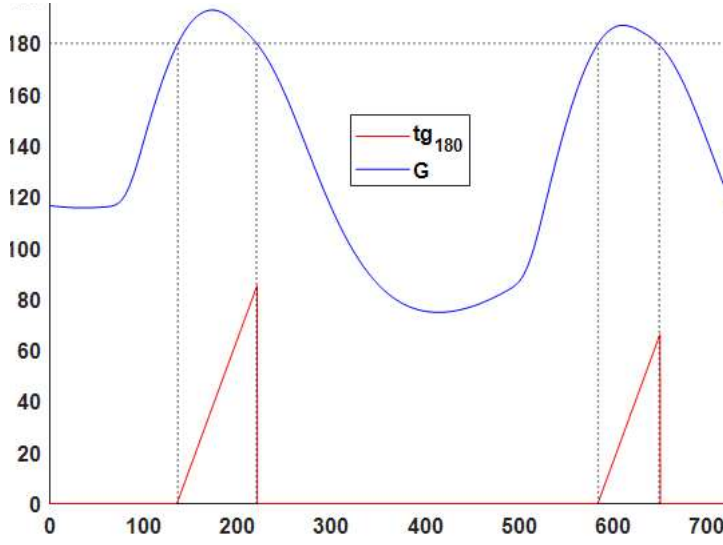


Fig. 21: The Duration of Time Where Glucose Exceeds 180 (mg/dL) $\{tg_{180}\}$

This property is satisfied if and only if a high glucose incidence would recover to normal range within two and a half hours maximum with at least 99% probability. It should be noted that the monitor PTA is constructed by creating a SysML activity diagram characterizing its behavior as shown in Fig. 22 and applying the new proposed automatic procedure to convert the EAC description into a PTA component that is parallel-composed with the other PTAs in UPPAAL-SMC tool.

$$\begin{aligned}
 Act_Monitor &= l \mapsto l_1 : B_C(l_2 : (C = G > 180) \mapsto N_1, l_3 : (C = G \leq 180) \mapsto N_2) \\
 N_1 &= l_4 : D_{CB}(G < 180, G \geq 180 - 1 \&\& tg'_{180} == 1) \\
 &\mapsto l_5 : Act(tg_{180} = 0) \mapsto l_6 : D_{CB}(G > 180 - 1, G \leq 180) \mapsto l_4 \\
 N_2 &= l_6
 \end{aligned}$$

The percentage of the patients with violations for each safety property is shown in Fig. 23. No violations exist in the absence of message errors. When message errors are introduced, the three control configurations result in varying behaviors. For safety property S_A , message errors result in a gradual increase of violations on *sustain* and *suspend* approaches. However, the *revert* approach preserves the safety property S_A on all patients with message errors up to 50%. For safety property S_B , the *suspend* approach fails on timely recovery of normal glucose levels in the existence of message errors. The other configurations, *sustain* and *revert*, avoid S_B violations with message errors as high as 30%. When the error rate exceeds that level, violations start to occur with the *revert* approach suffering more violations.

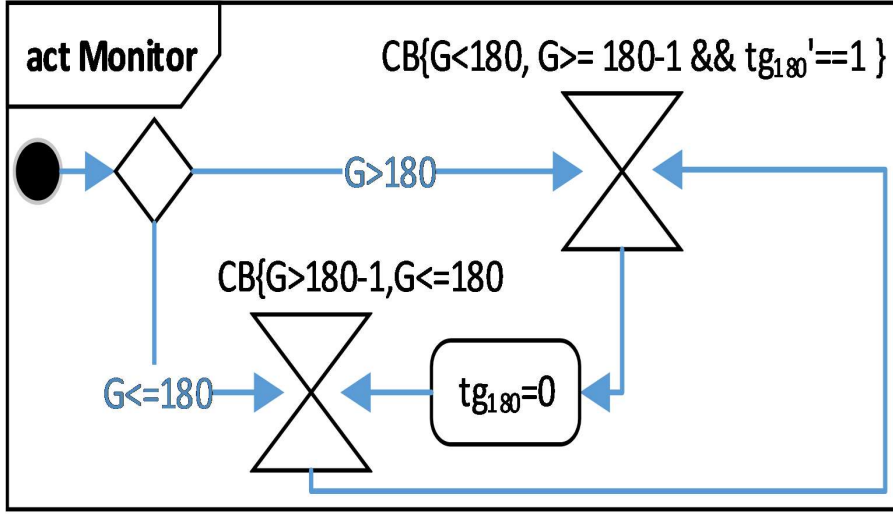


Fig. 22: SysML Activity Diagram of the Monitor

5.3 Discussion

To understand the experimental results, the following facts should be noted.

- In the absence of message errors, the three control configurations fall back to being the same standard PID controller.
- The analyzed artificial pancreas is a single hormone unidirectional controller (as opposed to dual-hormone systems [22]). This implies that it can deliver more insulin to counteract the excessive glucose levels, but it can only counteract low glucose levels by suspending the insulin delivery and waiting for the pre-delivered insulin to get consumed by the physiological processes inside the body.

Putting this in mind can explain the results on safety property S_A (left graph in Fig. 23), where the *sustain* approach accidentally delivers excessive insulin amounts that can cause glucose drops below 50 *mg/dL* even at low message error rates. On the contrary, the *suspend* approach stops insulin delivery and can make it up by restarting insulin delivery when valid messages are received again. However, when the message error rate increases, there is a chance that the *suspend* approach might fail to prevent large glucose levels above 300 *mg/dL*. Instead of completely halting the insulin delivery, the *revert* continues delivering small amounts of insulin to make a balance between the two other approaches and avoid extreme highs and lows of glucose. The same concept explains the results in the right graph of Fig. 23 where the *sustain* approach provides better performance in avoiding long times with glucose levels above 180 *mg/dL* as opposed to the *suspend* approach which fails to avoid that. The *revert* approach provides performance similar to the *sustain* approach

except for high message error rates where the violations start to increase when utilizing the *revert* approach.



Fig. 23: Results for Safety Properties Violations: S_A (left) and S_B (right)

6 Conclusion

In this work, a framework is proposed to formally model and automatically analyze cyber-physical systems using statistical model checking. The framework takes models specified using SysML modeling language as SysML diagrams. The latters are then represented in textual format using the proposed enhanced activity calculus and ordinary-differential equations of SysML constraint diagrams. Then, these textual representations of the model components are fed into a new proposed conversion algorithm that automatically transforms them into equivalent priced timed automata. Thus, the resulting model is fed into UPPAAL-SMC statistical model checking tool which parallel-composes all the system components and verifies the system behaviors. The use of the proposed framework to verify safety properties is demonstrated on an artificial pancreas case study.

The proposed framework can be used to verify the safety of cyber-physical systems and gain insight into their most critical behaviors at an early stage of the design process, thus saving valuable time and money. Ultimately, it promotes the integration of real-life problems into model-based analysis and allows experimenting a variety of scenarios without compromising participant safety. This is especially crucial when dealing with systems that involve human life, whether directly as in biomedical systems or indirectly as in automotive systems. In the near future, we target to improve the framework to cover more issues, mainly:

- Develop a library of different CPS components and applications.
- Model more cyber-physical systems with a focus on faults and security threats.
- Before the CPS deployment, we target also to automatically generate the source code related to the modeled and analyzed CPS.

- Provide guidance to correct the CPS whenever a property has not been satisfied.
- Establish a mechanism for defining CPS complex requirements automatically and easily.

References

1. Abdel-Latif Alshalalfah and Otmane Ait Mohamed. System-level modeling and safety analysis of vehicular coordinated emergency braking under degraded wireless connectivity using priced timed automata. In *2020 27th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, pages 1–4. IEEE, 2020.
2. Abdel-Latif Alshalalfah, Ghaith Bany Hamad, and Otmane Ait Mohamed. Towards system level security analysis of artificial pancreas via uppaal-smc. In *2019 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE, 2019.
3. Abdel-Latif Alshalalfah, Ghaith Bany Hamad, and Otmane Ait Mohamed. System-level analysis of closed-loop anesthesia control under temporal sensor faults via uppaal-smc. In *2020 42nd Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC)*, pages 2508–2511. IEEE, 2020.
4. Abdel-Latif Alshalalfah, Ghaith Bany Hamad, and Otmane Ait Mohamed. Towards safe and robust closed-loop artificial pancreas using improved pid-based control strategies. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 68(8):3147–3157, 2021.
5. Georges M Arnaout and Jean-Paul Arnaout. Exploring the effects of cooperative adaptive cruise control on highway traffic flow using microscopic traffic simulation. *Transportation Planning and Technology*, 37(2):186–199, 2014.
6. Ananda Basu, Saddek Bensalem, Marius Bozga, Benoît Delahaye, and Axel Legay. Statistical abstraction and model-checking of large heterogeneous systems. *International Journal on Software Tools for Technology Transfer*, 14(1):53–72, 2012.
7. Gerd Behrmann, Alexandre David, and Kim G Larsen. A tutorial on uppaal. *Formal methods for the design of real-time systems*, pages 200–236, 2004.
8. Peter Bulychyev, Alexandre David, Kim Guldstrand Larsen, Axel Legay, Guangyuan Li, Danny Bøgsted Poulsen, and Amélie Stainer. Monitor-based statistical model checking for weighted metric temporal logic. In *International Conference on Logic for Programming Artificial Intelligence and Reasoning*, pages 168–182. Springer, 2012.
9. Fraser Cameron, Georgios Fainekos, David M Maahs, and Sriram Sankaranarayanan. Towards a verified artificial pancreas: Challenges and solutions for runtime verification. In *Runtime Verification*, pages 3–17. Springer, 2015.

10. Edmund M Clarke Jr, Orna Grumberg, Daniel Kroening, Doron Peled, and Helmut Veith. *Model checking*. MIT press, 2018.
11. ML Cummings and David Britton. Regulating safety-critical autonomous systems: past, present, and future perspectives. In *Living with robots*, pages 119–140. Elsevier, 2020.
12. Alexandre David, Kim G Larsen, Axel Legay, Marius Mikučionis, and Zheng Wang. Time for statistical model checking of real-time systems. In *International Conference on Computer Aided Verification*, pages 349–355. Springer, 2011.
13. Alexandre David, DeHui Du, Kim G Larsen, Marius Mikučionis, and Arne Skou. An evaluation framework for energy aware buildings using statistical model checking. *Science China information sciences*, 55(12):2694–2707, 2012.
14. Alexandre David, Kim Guldstrand Larsen, Axel Legay, Marius Mikučionis, Danny Bøgsted Poulsen, and Sean Sedwards. Runtime verification of biological systems. In *International Symposium On Leveraging Applications of Formal Methods, Verification and Validation*, pages 388–404. Springer, 2012.
15. Alexandre David, Kim G Larsen, Axel Legay, Marius Mikučionis, and Danny Bøgsted Poulsen. Uppaal smc tutorial. *International journal on software tools for technology transfer*, 17(4):397–415, 2015.
16. Yair Bar David, Tal Geller, Ilai Bistriz, Irad Ben-Gal, Nicholas Bambos, and Evgeni Khmelnitsky. Wireless body area network control policies for energy-efficient health monitoring. *Sensors*, 21(12):4245, 2021.
17. Mourad Debbabi, Fawzi Hassaine, Yosr Jarraya, Andrei Soeanu, and Luay Alawneh. *Verification and validation in systems engineering: assessing UML/SysML design models*. Springer Science & Business Media, 2010.
18. Predrag Filipovikj, Nesredin Mahmud, Raluca Marinescu, Cristina Secleanu, Oscar Ljungkrantz, and Henrik Lönn. Simulink to uppaal statistical model checker: Analyzing automotive industrial systems. In *FM 2016: Formal Methods: 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings 21*, pages 748–756. Springer, 2016.
19. Goran Frehse. An introduction to hybrid automata, numerical simulation and reachability analysis. In *Formal Modeling and Verification of Cyber-Physical Systems: 1st International Summer School on Methods and Tools for the Design of Digital Systems, Bremen, Germany, September 2015*, pages 50–81. Springer, 2015.
20. Goran Frehse, Colas Le Guernic, Alexandre Donzé, Scott Cotton, Rajarshi Ray, Olivier Lebeltel, Rodolfo Ripado, Antoine Girard, Thao Dang, and Oded Maler. Spaceex: Scalable verification of hybrid systems. In *International Conference on Computer Aided Verification*, pages 379–395. Springer, 2011.
21. Patrice Godefroid. *Partial-order methods for the verification of concurrent systems: an approach to the state-explosion problem*. Springer, 1996.
22. Ahmad Haidar, Laurent Legault, Virginie Messier, Tina Maria Mitre, Catherine Leroux, and Rémi Rabasa-Lhoret. Comparison of dual-hormone

- artificial pancreas, single-hormone artificial pancreas, and conventional insulin pump therapy for glycaemic control in patients with type 1 diabetes: an open-label randomised controlled crossover trial. *The lancet Diabetes & endocrinology*, 3(1):17–26, 2015.
23. Jinpei Han, Joseph Davids, Hutan Ashrafian, Ara Darzi, Daniel S Elson, and Mikael Sodergren. A systematic review of robotic surgery: From supervised paradigms to fully autonomous robotic approaches. *The International Journal of Medical Robotics and Computer Assisted Surgery*, 18(2):e2358, 2022.
 24. Thomas Hérault, Richard Lassaigne, Frédéric Magniette, and Sylvain Peyronnet. Approximate probabilistic model checking. In *International Workshop on Verification, Model Checking, and Abstract Interpretation*, pages 73–84. Springer, 2004.
 25. Jon Holt and Simon Perry. *SysML for systems engineering*, volume 7. IET, 2008.
 26. Zhihao Jiang, Miroslav Pajic, Salar Moarref, Rajeev Alur, and Rahul Mangharam. Modeling and verification of a dual chamber implantable pacemaker. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 188–203. Springer, 2012.
 27. Nikolaos Kekatos, Marcelo Forets, and Goran Frehse. Constructing verification models of nonlinear simulink systems via syntactic hybridization. In *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, pages 1788–1795. IEEE, 2017.
 28. Nikolaos Kekatos, Marcelo Forets, and Goran Frehse. Modeling the wind turbine benchmark with pwa hybrid automata. *EPiC Series in Computing*, 48:100–113, 2017.
 29. Jun Kit Koong, Gaik Huey Ng, Kamarajan Ramayah, Peng Soon Koh, and Boon Koon Yoong. Early identification of the critical view of safety in laparoscopic cholecystectomy using indocyanine green fluorescence cholangiography: A randomised controlled study. *Asian Journal of Surgery*, 44(3):537–543, 2021.
 30. Ray Kurzweil. The law of accelerating returns. In *Alan Turing: Life and legacy of a great thinker*, pages 381–416. Springer, 2004.
 31. M. Kwiatkowska, G. Norman, and D. Parker. PRISM 4.0: Verification of probabilistic real-time systems. In G. Gopalakrishnan and S. Qadeer, editors, *Proc. 23rd International Conference on Computer Aided Verification (CAV’11)*, volume 6806 of *LNCS*, pages 585–591. Springer, 2011.
 32. Sridhar Lakshmanan, Yuedong Yan, Stan Baek, and Hesham Alghodhaifi. Modeling and simulation of leader-follower autonomous vehicles: environment effects. In *Unmanned systems technology XXI*, volume 11021, page 110210J. International Society for Optics and Photonics, 2019.
 33. Kim G Larsen and Arne Skou. Bisimulation through probabilistic testing. *Information and computation*, 94(1):1–28, 1991.
 34. Srinivas Laxminarayan, Jaques Reifman, and Garry M Steil. Use of a food and drug administration-approved type 1 diabetes mellitus simulator to evaluate and optimize a proportional-integral-derivative controller.

- Journal of diabetes science and technology*, 6(6):1401–1412, 2012.
35. Axel Legay, Anna Lukina, Louis Marie Traonouez, Junxing Yang, Scott A Smolka, and Radu Grosu. Statistical model checking. In *Computing and Software Science*, pages 478–504. Springer, 2019.
 36. Alexios Lekidis, Paraskevas Bourgos, Simplicio Djoko-Djoko, Marius Bozga, and Saddek Bensalem. Building distributed sensor network applications using bip. In *2015 IEEE Sensors Applications Symposium (SAS)*, pages 1–6. IEEE, 2015.
 37. Jun Liu, Kara M Kockelman, Patrick M Boesch, and Francesco Ciari. Tracking a system of shared autonomous vehicles across the austin, texas network using agent-based simulation. *Transportation*, 44(6):1261–1278, 2017.
 38. Chiara Dalla Man, Francesco Micheletto, Dayu Lv, Marc Breton, Boris Kovatchev, and Claudio Cobelli. The uva/padova type 1 diabetes simulator: new features. *Journal of diabetes science and technology*, 8(1):26–34, 2014.
 39. Braham Lotfi Mediouni, Ayoub Nouri, Marius Bozga, Mahieddine Dellabani, Axel Legay, and Saddek Bensalem. SBIP 2.0: Statistical model checking stochastic real-time systems. In *International Symposium on Automated Technology for Verification and Analysis*, pages 536–542. Springer, 2018.
 40. Stefano Minopoli and Goran Frehse. Sl2sx translator: from simulink to spacex models. In *Proceedings of the 19th International Conference on Hybrid Systems: Computation and Control*, pages 93–98, 2016.
 41. Umberto Montanaro, Shilp Dixit, Saber Fallah, Mehrdad Dianati, Alan Stevens, David Oxtoby, and Alexandros Mouzakitis. Towards connected autonomous driving: review of use-cases. *Vehicle system dynamics*, 57(6):779–814, 2019.
 42. Katherine Ogurtsova, JD da Rocha Fernandes, Y Huang, Ute Linnenkamp, L Guariguata, Nam H Cho, David Cavan, JE Shaw, and LE Makaroff. Idf diabetes atlas: Global estimates for the prevalence of diabetes for 2015 and 2040. *Diabetes research and clinical practice*, 128:40–50, 2017.
 43. Masashi Okamoto. Some inequalities relating to the partial sum of binomial probabilities. *Annals of the institute of Statistical Mathematics*, 10(1):29–35, 1959.
 44. Samir Ouchani, Yosr Jarraya, Otmane Ait Mohamed, and Mourad Debbabi. Probabilistic attack scenarios to evaluate policies over communication protocols. *J. Softw.*, 7(7):1488–1495, 2012.
 45. Samir Ouchani, Otmane Ait Mohamed, and Mourad Debbabi. A formal verification framework for sysml activity diagrams. *Expert Systems with Applications*, 41(6):2713–2728, 2014.
 46. Miroslav Pajic, Rahul Mangharam, Oleg Sokolsky, David Arney, Julian Goldman, and Insup Lee. Model-driven safety analysis of closed-loop medical systems. *IEEE Transactions on Industrial Informatics*, 10(1):3–16, 2012.

- 1007 47. Alkis Papadoulis, Mohammed Quddus, and Marianna Imprialou. Evaluat-
1008 ing the safety impact of connected and autonomous vehicles on motorways.
1009 *Accident Analysis & Prevention*, 124:12–22, 2019.
- 1010 48. Sriram Sankaranarayanan, Suhas Akshar Kumar, Faye Cameron, B Wayne
1011 Bequette, Georgios Fainekos, and David M Maahs. Model-based falsifica-
1012 tion of an artificial pancreas control system. *ACM SIGBED Review*, 14
1013 (2):24–33, 2017.
- 1014 49. Stefan Schupp, Erika Ábrahám, Xin Chen, Ibtissem Ben Makhoulf, Goran
1015 Frehse, Sriram Sankaranarayanan, and Stefan Kowalewski. Current chal-
1016 lenges in the verification of hybrid systems. In *International Workshop on*
1017 *Design, Modeling, and Evaluation of Cyber Physical Systems*, pages 8–24.
1018 Springer, 2015.
- 1019 50. Stefan Schupp, Francesco Leofante, Leander Behr, Erika Ábrahám, and
1020 Armando Taccella. Robot swarms as hybrid systems: Modelling and veri-
1021 fication. *arXiv preprint arXiv:2207.06758*, 2022.
- 1022 51. Koushik Sen, Mahesh Viswanathan, and Gul Agha. Statistical model
1023 checking of black-box probabilistic systems. In *International Conference*
1024 *on Computer Aided Verification*, pages 202–215. Springer, 2004.
- 1025 52. Robert E Shannon. Systems simulation; the art and science. Technical
1026 report, 1975.
- 1027 53. Alyona Skorobogatjko, Andrejs Romanovs, and Nadezhda Kunicina. State
1028 of the art in the healthcare cyber-physical systems. *Information Technol-*
1029 *ogy and Management Science*, 17(1):126–131, 2014.
- 1030 54. OMG Available Specification. Omg systems modeling language (omg
1031 sysmlTM), v1. 0, 2007.
- 1032 55. Kay W Axhausen, Andreas Horni, and Kai Nagel. *The multi-agent trans-*
1033 *port simulation MATSim*. Ubiquity Press, 2016.
- 1034 56. Abraham Wald. *Sequential analysis*. Courier Corporation, 2004.
- 1035 57. Hakan Lorens Samir Younes. *Verification and planning for stochastic pro-*
1036 *cesses with asynchronous events*. Carnegie Mellon University, 2004.
- 1037 58. Håkan LS Younes and Reid G Simmons. Probabilistic verification of dis-
1038 crete event systems using acceptance sampling. In *International Confer-*
1039 *ence on Computer Aided Verification*, pages 223–235. Springer, 2002.